TRIMESTER OCTOBER/NOVEMBER 2024

TDB6113 DATABASE SYSTEMS

Online Course Management System

PROJECT PREPARED BY

NAME1:	Bin Thabit Hussein Abbod	ID 241UT240SP	AI
NAME2:	Yousef. Rian Ali	ID 241UT24010	AI
NAME3:	Mohammed Abdulsalam Ahmed	ID 241UT240M7	AI
NAME4:	Jaeran, Osamah Nabil Saleh ali	ID 241UT240M8	AI

LAB SECTION: 1D

FACULTY OF INFORMATION SCIENCE & TECHNOLOGY

MULTIMEDIA UNIVERSITY
MALAYSIA
27 JANUARY 2025

TABLE OF CONTENTS

Table of Contents

1 Introduction: 4
2 Problem Statements & Objectives 5
1 Business Rules 6
4 Entity Relationship Diagram 7
5 Normalization 8
6 Data Dictionary 11
STUDENT Table 11
ENROLLMENT Table 12
COURSE Table 13
INSTRUCTOR Table 14
ASSIGNMENT Table 14
SUBMISSION Table 15
LEARNING MATERIAL Table 15
7 SQL/Screenshot 16
1. Table Creation 16
2. Foreign Key Constraints 22
3. Data Insertion 25
4: Trigger: 30
Trigger 1: Prevent duplicate enrollment: 31
Trigger 2. Prevent Invalid Enrollment Dates 32
Trigger 3: Prevent Assignment Submission After Due Date: 34
5: Function 35
Function 1: Calculate average grade for a course: 36

Function 2: Get total payments made by a student: 38
Function 3: Count total students in a course: 39
Function 4: Check if material is accessible by permission: 41
Function 5: Calculate days left for assignment deadline: 43
6: Procedure 44
Procedure 1: Enroll a student into a course 44
Procedure 2: Submit an assignment 46
Procedure 3: Update course fees 49
Procedure 4: drop student from course; 50
7: TEST: 53
8 Business Ideas 62
Revolutionizing Education Through Digital Integration 62
AI-driven Personalized Learning Platform 63
Blockchain-Based Digital Identity Verification Platform 65
Online Learning Platform for Small and Medium-Sized Educational Institutions 67
Corporate Training and Development Platform 70
10 references 74

1 Introduction:

This revolution in technologies has changed the contours of education, making it more flexible, approachable, and adaptive to modern society's requirements. Traditional course management systems are fraught with problems: long manual processing, inefficient tracking of student performance, and non-integrated academic processes—these have adverse implications for administrative efficiency and interest among students in their studies.

To these challenges the E-Learning Course Management System responds with an integrated database-driven approach. It is based on five major entities constituting the core in the operation of this system:

1 Courses: This module is designed to manage every detail related to courses It includes the course title a comprehensive description outlining the course objectives, a detailed schedule specifying the timeline and key milestones and a list of resources required for effective learning. This ensures that both students and instructors have clear and organized access to essential course information

2 Students: This module captures and maintains comprehensive records for each student It includes personal information such as name contact details and enrollment ID as well as academic progress including grades completed courses and current enrollments Additionally it tracks enrollment history to provide a complete academic profile of the student

3.Instructors: This module stores detailed profiles of instructors including their personal information expertise and courses taught It also includes functionality for instructors to assess and record the performance of students in their courses providing valuable insights into teaching outcomes and student learning progress

4.Assignments: This module facilitates the management of assignments from creation to grading It includes functionality to set submission deadlines track student submissions and provide grades and feedback By automating these processes it ensures timely and efficient handling of assignments for both students and instructors

5 Learning Materials: This module provides a centralized catalog for all educational materials associated with the courses It allows instructors to upload and manage resources such as lecture notes videos and reading materials Students can easily access these resources ensuring they have the necessary tools to succeed in their studies

2 Problem Statements & Objectives

Problem Statement	Objectives
Inefficient Course Management	Manual effort in the traditional systems to manage courses regarding scheduling, updating materials, and tracking timelines results in a lot of inefficiency, and hence it diminishes administrative productivity and creates confusion for students and instructors.
Fragmented Student and Instructor Data	Many of the existing academic systems store records of students and instructors separately and are unlinked, which causes redundancy and errors and will lead to inadequate traceability of academic progress and teaching outcomes.
Limited Accessibility to Learning Resources	With so many scattered learning materials, students usually find it hard to access the centralized repository of learning materials. This results in poor preparation and less involvement in their studies.
Streamline Course Management	Create a single, fully integrated module to manage courses—planning, resources,
Centralize and Optimize Data Handling	Create a robust database that will store and connect detailed profiles of both students and instructors, allowing for seamless tracking of academic and
More Resources at Your Fingertips	Centralized, easy-to-use learning material catalog accessible to both students

1 Business Rules

1. Many-to-Many (M:N) Relationship Between Students and Courses

A student can enrols in multiple courses, and a course can have multiple students enrolled.

2. One-to-Many (1:M) Relationship Between Instructors and Courses

An instructor can teach multiple courses, but each course is assigned to only one instructor.

3. One-to-Many(1:M) Relationship Between Courses and Assignments

A course can include multiple assignments, with each assignment being exclusively associated with a single course.

4. Many-to-Many (M:N) Relationship Between Students and Assignments

A student can submit multiple assignments, and an assignment can be submitted by multiple students

5. One-to-Many (1:M) Relationship Between Courses and Learning Materials

A course can have multiple learning materials associated with it, but each learning material is linked to only one course.

4 Entity Relationship Diagram

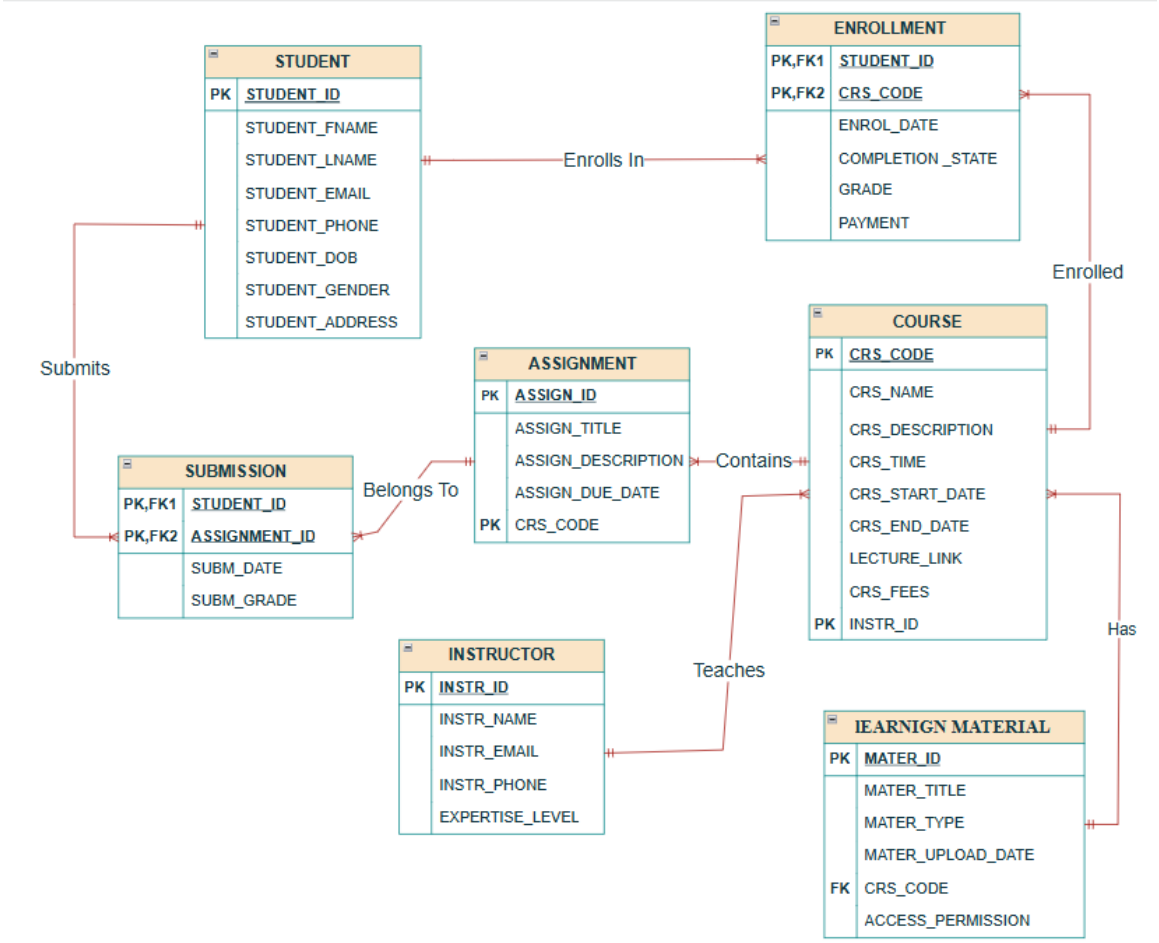

Figure 1:ERD

5 Normalization

Normalization Process

1st Normal Form (1NF)

In the 1st normalization process, we identify the initial entities and their attributes. The goal is to eliminate repeating groups and ensure atomicity (each column contains only one value).

Student (**Student_ID (PK)**, First_Name, Last_Name, Email, Phone, DOB, Gender, Address)

Course (**Course_ID (PK),** Title, Description, Expertise_level)

Enrollment (**Enrollment_ID (PK),** Student_ID (FK), Course_ID (FK), Enrollment_Date)

Assignment (**Assignment_ID (PK)**, Title, Description, Due_Date, Course_ID (FK))

Instructor: **(INSTR_ID**, INSTR_NAME, INSTR_EMAIL, INSTR_PHONE, EXPERTISE_LEVEL)

Learning_Materials (**Material_ID (PK),** Title, Type, Upload_Date, Course_ID (FK), Access_Permissions)

Enrolled (**Enrolled_ID (PK),** Student_ID (FK), Course_ID (FK), Enrollment_Date)

In this 1NF schema:

- Each table has a primary key.
- All attributes are atomic.
- Redundant data has been removed by creating separate tables for entities like Student, Course, and Instructor.
- Foreign keys are used to establish relationships between tables.

This structure ensures that each table represents a single entity or relationship, and each attribute contains only indivisible values.

2nd Normal Form (2NF)

In the 2nd normalization process, we remove partial dependencies and ensure that all non-key attributes are fully dependent on the primary key.

1. The schema is already in **1NF**.
2. All non-key attributes are **fully functionally dependent** on the **primary key** (i.e., there should be no partial dependencies).

Changes Made for 2NF:

1. **Removed Partial Dependencies**: In the original schema, some tables had attributes that were not fully dependent on the primary key. For example, in the "Enrolls In" and "Enrollment" tables, attributes like First_Name, Last_Name, Email, and Phone were repeated and partially dependent on the primary key. These were removed and replaced with foreign keys (Student_ID, Course_ID) to ensure full dependency.
2. **Normalized Repeated Attributes**: Attributes like First_Name, Last_Name, Email, and Phone were moved to their respective tables (Student, Teacher) to avoid redundancy and ensure full dependency on the primary key.

3rd Normal Form (3NF)

In the 3rd normalization process, we remove transitive dependencies and ensure that all attributes are dependent only on the primary key.

1. The schema is already in **2NF**.
2. There are **no transitive dependencies** (i.e., non-key attributes should not depend on other non-key attributes).

Changes Made for 3NF:

1. **Removed Transitive Dependencies**: In the original schema, there were no clear transitive dependencies because all non-key attributes were directly dependent on the primary key. However, we ensured that no non-key attribute depends on another non-key attribute.

2. **Normalized Further**: If any transitive dependencies were found (e.g., if an attribute like Expertise_level depended on another non-key attribute), we would have split the table further. In this case, no such dependencies exist.

Final Tables after Normalization

- **Student**: (**Student_ID** , Student_FName, Student_LName, Student_Email, Student_Phone, Student_DOB, Student_Gender, Student_Address)
- **Enrollment**: (**Student_ID(PK, FK),CRS_CODE(PK,FK)**, ENROLL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
- **Submission**: **(Student_ID (PK, FK), ASSIGN_ID (PK, FK)**, SUBM_DATE, SUBM_GRADE)
- **Assignment**: (**ASSIGN_ID**, ASSIGN_TITLE, ASSIGN_DESCRIPTION, ASSIGN_DUE_DATE, CRS_CODE (FK))
- **Course**: **(CRS_CODE**, CRS_NAME, CRS_DESCRIPTION, CRS_TIME, CRS_START_DATE, CRS_END_DATE, LECTURE_LINK, CRS_FEES, INSTR_ID (FK))
- **Instructor**: **(INSTR_ID**, INSTR_NAME, INSTR_EMAIL, INSTR_PHONE, EXPERTISE_LEVEL)
- **Teaching Material**: **(MATER_ID**, MATER_TITLE, MATER_TYPE, MATER_UPLOAD_DATE, CRS_CODE (FK), ACCESS_PERMISSION)

6 Data Dictionary

STUDENT Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
STUDENT_ID	Unique identifier	INT			Y	PK	
STUDENT_FNAME	First name	NVARCHAR (50)	Xxxxxx		Y		
STUDENT_LNAME	Last name	NVARCHAR (50)	Xxxxxx		Y		
STUDENT_EMAIL	Email address	NVARCHAR(100)	Email		Y		
STUDENT_PHONE	Phone number	NVARCHAR (15)	999999999		N		
STUDENT_DOB	Date of birth	DATE	YYYY-MM-DD		N		
STUDENT_GENDER	Gender	CHAR (1)	Male/Female		N		
STUDENT_ADDRESS	Address	NVARCHAR(255)	Xxxxxx		N		

ENROLLMENT Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
STUDENT_ID	Reference to student	INT			Y	FK	STUDENT
CRS_CODE	Reference to course	INT			Y	FK	COURSE
ENROL_DATE	Enrollment date	DATE	YYYY-MM-DD		Y		
COMPLETION_STATE	Completion status	NVARCHAR(50)	Xxxxxx		N		
GRADE	Final grade	NVARCHAR (2)	X		N		
PAYMENT	Payment status	NVARCHAR(50)	Paid/Unpaid		N		

COURSE Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
CRS_CODE	Unique course code	INT			Y	PK	
CRS_NAME	Course name	NVARCHAR(100)	Xxxxxx		Y		
CRS_DESCRIPTION	Description	NVARCHAR(255)	Xxxxxx		N		
CRS_TIME	Duration (hrs)	NVARCHAR(50)	99		Y		
CRS_START_DATE	Start date	DATE	YYYY-MM-DD		Y		
CRS_END_DATE	End date	DATE	YYYY-MM-DD		Y		
LECTURE_LINK	Lecture link	NVARCHAR(255)	URL		N		
CRS_FEES	Fees	FLOAT	9999.99	0.00 9999.99	Y		
INSTR_ID	Instructor reference	INT		1-100000	Y	FK	INSTRUCTOR

INSTRUCTOR Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
INSTR_ID	Instructor ID	INT		1-100000	Y	PK	
INSTR_NAME	Name	NVARCHAR(100)	Xxxxxx		Y		
INSTR_EMAIL	Email	NVARCHAR(100)	Email		Y		
INSTR_PHONE	Phone number	NVARCHAR(15)	999999999		N		
EXPERTISE_LEVEL	Expertise	VARCHAR(50)	Xxxxxx		N		

ASSIGNMENT Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
ASSIGN_ID	Assignment ID	INT		1-100000	Y	PK	
ASSIGN_TITLE	Title	NVARCHAR(100)	Xxxxxx		Y		
ASSIGN_DESCRIPTION	Description	TEXT	Xxxxxx		N		
ASSIGN_DUE_DATE	Due date	DATE	YYYY-MM-DD		Y		
CRS_CODE	Course reference	VARCHAR(10)	Xxxxxx		Y	FK	COURSE

SUBMISSION Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
STUDENT_ID	Student reference	INT			Y	FK	STUDENT
ASSIGNMENT_ID	Assignment reference	INT			Y	FK	ASSIGNMENT
SUBM_DATE	Submission date	DATE	YYYY-MM-DD		Y		
SUBM_GRADE	Grade	NVARCHAR(2)	X	0-100	N		

LEARNING MATERIAL Table

ATTRIBUTE NAME	CONTENTS	TYPE	FORMAT	RANGE	REQUIRED	PK OR FK	REFERENCED TABLE
MATER_ID	Material ID	INT		1-100000	Y	PK	
MATER_TITLE	Title	NVARCHAR(100)	Xxxxxx		Y		
MATER_TYPE	Type	NVARCHAR(50)	Xxxxxx		Y		
MATER_UPLOAD_DATE	Upload date	DATE	YYYY-MM-DD		Y		
CRS_CODE	Course reference	INT			Y	FK	COURSE
ACCESS_PERMISSION	Permission	NVARCHAR(50)	Xxxxxx		N		

7 SQL/Screenshot

```
MariaDB [(none)]> create database Online_Course;
Query OK, 1 row affected (0.004 sec)

MariaDB [(none)]> |
```

```
MariaDB [(none)]> use Online_Course;
Database changed
MariaDB [Online_Course]> |
```

For any database, the primary essential task is to create tables. Tables act like containers, including management, which stores different types of data. Understanding the purpose behind each table will be even more helpful:

1. <u>Table Creation</u>

- *STUDENT Table: This is to store all the crucial details of the students such as ID Student_ID, name, age, phone number, date of birth, sex, and address.*

```
-- Create a table to store student information
CREATE TABLE STUDENT (
   STUDENT_ID INT PRIMARY KEY,                -- Unique identifier for each student (Primary Key)
   STUDENT_FNAME NVARCHAR(50),            -- Student's first name
   STUDENT_LNAME NVARCHAR(50),            -- Student's last name
   STUDENT_EMAIL NVARCHAR(100),           -- Student's email address
   STUDENT_PHONE NVARCHAR(15),            -- Student's phone number
   STUDENT_DOB DATE,                      -- Student's date of birth
   STUDENT_GENDER CHAR(1),                -- Student's gender (e.g., 'M' for Male, 'F' for Female)
   STUDENT_ADDRESS NVARCHAR(255)          -- Student's residential address
```

);

Output

```
MariaDB [Online_Course]> -- Create a table to store student information
MariaDB [Online_Course]> CREATE TABLE STUDENT (
    ->     STUDENT_ID INT PRIMARY KEY,              -- Unique identifier for each student (Primary Key)
    ->     STUDENT_FNAME NVARCHAR(50),              -- Student's first name
    ->     STUDENT_LNAME NVARCHAR(50),              -- Student's last name
    ->     STUDENT_EMAIL NVARCHAR(100),             -- Student's email address
    ->     STUDENT_PHONE NVARCHAR(15),              -- Student's phone number
    ->     STUDENT_DOB DATE,                        -- Student's date of birth
    ->     STUDENT_GENDER CHAR(1),                  -- Student's gender (e.g., 'M' for Male, 'F' for Female)
    ->     STUDENT_ADDRESS NVARCHAR(255)            -- Student's residential address
    -> );
Query OK, 0 rows affected (0.031 sec)
```

- *INSTRUCTOR Table: In this table the instructors, their ID, name, expertise, email, and contact number would be stored. The primary feature is INSTR_ID because every instructor is unique.*

-- Create a table to store instructor information

CREATE TABLE INSTRUCTOR (

INSTR_ID INT PRIMARY KEY, -- Unique identifier for each instructor (Primary Key)

INSTR_NAME NVARCHAR(100), -- Instructor's full name

INSTR_EMAIL NVARCHAR(100), -- Instructor's email address

INSTR_PHONE NVARCHAR(15), -- Instructor's phone number

EXPERTISE_LEVEL NVARCHAR(50) -- Instructor's level of expertise (e.g., Beginner, Intermediate, Expert)

);

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a table to store instructor information
MariaDB [Online_Course]> CREATE TABLE INSTRUCTOR (
    ->     INSTR_ID INT PRIMARY KEY,                -- Unique identifier for each instructor (Primary Key)
    ->     INSTR_NAME NVARCHAR(100),                -- Instructor's full name
    ->     INSTR_EMAIL NVARCHAR(100),               -- Instructor's email address
    ->     INSTR_PHONE NVARCHAR(15),                -- Instructor's phone number
    ->     EXPERTISE_LEVEL NVARCHAR(50)             -- Instructor's level of expertise (e.g., Beginner, Intermediate, Expert)

    -> );
Query OK, 0 rows affected (0.009 sec)
```

- COURSE Table: This table contains information about the subject, such as course code, course name, course details, schedule, enrollment start date, the ending date of lectures, lecture link, course fee, and course instructor. The primary key is the CRS_CODE, and the INSTR_ID is a foreign key corresponding to the INSTRUCTOR table.

```
-- Create a table to store course information
CREATE TABLE COURSE (
   CRS_CODE INT PRIMARY KEY,            -- Unique identifier for each course (Primary Key)
   CRS_NAME NVARCHAR(100),              -- Name of the course
   CRS_DESCRIPTION NVARCHAR(255),       -- Description of the course
   CRS_TIME NVARCHAR(50),               -- Scheduled time for the course (e.g., "10:00 AM - 12:00 PM")
   CRS_START_DATE DATE,                 -- Start date of the course
   CRS_END_DATE DATE,                   -- End date of the course
   LECTURE_LINK NVARCHAR(255),          -- Link to access course lectures (e.g., Zoom or Google Meet link)
   CRS_FEES FLOAT,                      -- Course fees
   INSTR_ID INT                         -- Foreign Key referencing the instructor teaching the course
);
```

Output

```
MariaDB [Online_Course]> -- Create a table to store course information
MariaDB [Online_Course]> CREATE TABLE COURSE (
    ->     CRS_CODE INT PRIMARY KEY,                -- Unique identifier for each course (Primary Key)
    ->     CRS_NAME NVARCHAR(100),                  -- Name of the course
    ->     CRS_DESCRIPTION NVARCHAR(255),           -- Description of the course
    ->     CRS_TIME NVARCHAR(50),                   -- Scheduled time for the course (e.g., "10:00 AM - 12:00 PM")
    ->     CRS_START_DATE DATE,                     -- Start date of the course
    ->     CRS_END_DATE DATE,                       -- End date of the course
    ->     LECTURE_LINK NVARCHAR(255),              -- Link to access course lectures (e.g., Zoom or Google Meet link)
    ->     CRS_FEES FLOAT,                          -- Course fees
    ->     INSTR_ID INT                             -- Foreign Key referencing the instructor teaching the course
    -> );
Query OK, 0 rows affected (0.009 sec)
```

- ENROLLMENT Table: This table will have information on the enrollment of students in certain courses given the student ID, course code, enrollment date, status of completion, grade, and payment status. The composite primary key of STUDENT_ID + CRS_CODE ensures a student can enroll in a specific course only once.

```
-- Create a table to store student enrollment information
CREATE TABLE ENROLLMENT (
   STUDENT_ID INT,                          -- Foreign Key referencing the student enrolled
   CRS_CODE INT,                            -- Foreign Key referencing the course enrolled in
   ENROL_DATE DATE,                         -- Date of enrollment
   COMPLETION_STATE NVARCHAR(50),           -- Enrollment completion status (e.g., "In Progress", "Completed")
   GRADE NVARCHAR(2),                       -- Grade achieved by the student in the course
   PAYMENT NVARCHAR(50),                    -- Payment status (e.g., "Paid", "Pending")
   PRIMARY KEY (STUDENT_ID, CRS_CODE)       -- Composite Primary Key (STUDENT_ID + CRS_CODE)
);
```

Output

```
MariaDB [Online_Course]> -- Create a table to store student enrollment information
MariaDB [Online_Course]> CREATE TABLE ENROLLMENT (
    ->     STUDENT_ID INT,                          -- Foreign Key referencing the student enrolled
    ->     CRS_CODE INT,                            -- Foreign Key referencing the course enrolled in
    ->     ENROL_DATE DATE,                         -- Date of enrollment
    ->     COMPLETION_STATE NVARCHAR(50),           -- Enrollment completion status (e.g., "In Progress", "Completed")
    ->     GRADE NVARCHAR(2),                       -- Grade achieved by the student in the course
    ->     PAYMENT NVARCHAR(50),                    -- Payment status (e.g., "Paid", "Pending")
    ->     PRIMARY KEY (STUDENT_ID, CRS_CODE)       -- Composite Primary Key (STUDENT_ID + CRS_CODE)
    -> );
Query OK, 0 rows affected (0.009 sec)
```

- ASSIGNMENT Table: This table keeps information on all assignments for each course like the assignment ID, title, description, due date, and the course it belongs to. The CRS_CODE is a foreign key to the COURSE table.

-- Create a table to store assignment details for courses

CREATE TABLE ASSIGNMENT (

ASSIGN_ID INT PRIMARY KEY, -- Unique identifier for each assignment (Primary Key)

ASSIGN_TITLE NVARCHAR(100), -- Title of the assignment

ASSIGN_DESCRIPTION NVARCHAR(255), -- Description of the assignment

ASSIGN_DUE_DATE DATE, -- Due date for the assignment

CRS_CODE INT -- Foreign Key referencing the course the assignment belongs to

);

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a table to store assignment details for courses
MariaDB [Online_Course]> CREATE TABLE ASSIGNMENT (
    ->     ASSIGN_ID INT PRIMARY KEY,              -- Unique identifier for each assignment (Primary Key)
    ->     ASSIGN_TITLE NVARCHAR(100),             -- Title of the assignment
    ->     ASSIGN_DESCRIPTION NVARCHAR(255),       -- Description of the assignment
    ->     ASSIGN_DUE_DATE DATE,                   -- Due date for the assignment
    ->     CRS_CODE INT                            -- Foreign Key referencing the course the assignment belongs to
    -> );
Query OK, 0 rows affected (0.008 sec)
```

- SUBMISSION Table: This table includes assignments for student submissions. That would include details like student ID, assignment ID, date of submission, and marks. The composite primary key for STUDENT_ID + ASSIGNMENT_ID makes sure the student can submit the same assignment once.

-- Create a table to store student submissions for assignments

CREATE TABLE SUBMISSION (

STUDENT_ID INT, -- Foreign Key referencing the student who submitted the assignment

ASSIGNMENT_ID INT, -- Foreign Key referencing the assignment submitted

SUBM_DATE DATE, -- Date of submission

SUBM_GRADE NVARCHAR(2), -- Grade awarded for the submission

PRIMARY KEY (STUDENT_ID, ASSIGNMENT_ID) -- Composite Primary Key (STUDENT_ID + ASSIGNMENT_ID)

);

Output

```
MariaDB [Online_Course]> -- Create a table to store student submissions for assignments
MariaDB [Online_Course]> CREATE TABLE SUBMISSION (
    ->     STUDENT_ID INT,                          -- Foreign Key referencing the student who submitted the assignment
    ->     ASSIGNMENT_ID INT,                       -- Foreign Key referencing the assignment submitted
    ->     SUBM_DATE DATE,                          -- Date of submission
    ->     SUBM_GRADE NVARCHAR(2),                  -- Grade awarded for the submission
    ->     PRIMARY KEY (STUDENT_ID, ASSIGNMENT_ID)  -- Composite Primary Key (STUDENT_ID + ASSIGNMENT_ID)
    -> );
Query OK, 0 rows affected (0.009 sec)
```

- LEARNING_MATERIAL Table: Learning materials (e.g., videos and pdfs) for each course are saved in the table. The table holds material ID, title, type, date uploaded, course_code, and access privileges. CRS_CODE is a foreign key with relation to the table in which it is referenced: the table named COURSE.

-- Create a table to store learning materials for courses

CREATE TABLE LEARNING_MATERIAL (

MATER_ID INT PRIMARY KEY, -- Unique identifier for each learning material (Primary Key)

MATER_TITLE NVARCHAR(100), -- Title of the learning material

MATER_TYPE NVARCHAR(50), -- Type of material (e.g., "PDF", "Video", "Slide")

MATER_UPLOAD_DATE DATE, -- Date the material was uploaded

CRS_CODE INT, -- Foreign Key referencing the course the material belongs to

ACCESS_PERMISSION NVARCHAR(50) -- Access permissions for the material (e.g., "Public", "Private")

);

Output

```
MariaDB [Online_Course]> -- Create a table to store learning materials for courses
MariaDB [Online_Course]> CREATE TABLE LEARNING_MATERIAL (
    ->     MATER_ID INT PRIMARY KEY,                -- Unique identifier for each learning material (Primary Key)
    ->     MATER_TITLE NVARCHAR(100),               -- Title of the learning material
    ->     MATER_TYPE NVARCHAR(50),                 -- Type of material (e.g., "PDF", "Video", "Slide")
    ->     MATER_UPLOAD_DATE DATE,                  -- Date the material was uploaded
    ->     CRS_CODE INT,                            -- Foreign Key referencing the course the material belongs to
    ->     ACCESS_PERMISSION NVARCHAR(50)           -- Access permissions for the material (e.g., "Public", "Private")
    -> );
Query OK, 0 rows affected (0.015 sec)
```

2. Foreign Key Constraints

Once tables have been created, foreign key constraints can be added to preserve relations in between tables. These constraints preserve integrity and consistency of data. Examples could include in terms of an instructor teaching a course that doesn't have a real record in the INSTRUCTOR table. Also, a student can't register for a course not present in the COURSE table, and an assignment can't be under a course when the course doesn't even exist.

Drop foreign key constraints.

-- Add a foreign key constraint to the COURSE table linking INSTR_ID to the INSTRUCTOR table

ALTER TABLE COURSE

ADD CONSTRAINT FK_COURSE_INSTRUCTOR

FOREIGN KEY (INSTR_ID) REFERENCES INSTRUCTOR(INSTR_ID);

-- Ensures that the INSTR_ID in the COURSE table must exist in the INSTRUCTOR table

Output

```
MariaDB [Online_Course]> -- Add a foreign key constraint to the COURSE table linking INSTR_ID to the INSTRUCTOR table
MariaDB [Online_Course]> ALTER TABLE COURSE
    -> ADD CONSTRAINT FK_COURSE_INSTRUCTOR
    -> FOREIGN KEY (INSTR_ID) REFERENCES INSTRUCTOR(INSTR_ID);
Query OK, 0 rows affected (0.027 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Add a foreign key constraint to the ENROLLMENT table linking STUDENT_ID to the STUDENT table

ALTER TABLE ENROLLMENT

ADD CONSTRAINT FK_ENROLLMENT_STUDENT

FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT(STUDENT_ID);

-- Ensures that the STUDENT_ID in the ENROLLMENT table must exist in the STUDENT table

Output

```
MariaDB [Online_Course]> -- Ensures that the INSTR_ID in the COURSE table must exist in the INSTRUCTOR table
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Add a foreign key constraint to the ENROLLMENT table linking STUDENT_ID to the STUDENT table
MariaDB [Online_Course]> ALTER TABLE ENROLLMENT
    -> ADD CONSTRAINT FK_ENROLLMENT_STUDENT
    -> FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT(STUDENT_ID);
Query OK, 0 rows affected (0.026 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Add a foreign key constraint to the ENROLLMENT table linking CRS_CODE to the COURSE table

ALTER TABLE ENROLLMENT

ADD CONSTRAINT FK_ENROLLMENT_COURSE

FOREIGN KEY (CRS_CODE) REFERENCES COURSE(CRS_CODE);

-- Ensures that the CRS_CODE in the ENROLLMENT table must exist in the COURSE table

Output

```
MariaDB [Online_Course]> -- Ensures that the STUDENT_ID in the ENROLLMENT table must exist in the STUDENT table
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Add a foreign key constraint to the ENROLLMENT table linking CRS_CODE to the COURSE table
MariaDB [Online_Course]> ALTER TABLE ENROLLMENT
    -> ADD CONSTRAINT FK_ENROLLMENT_COURSE
    -> FOREIGN KEY (CRS_CODE) REFERENCES COURSE(CRS_CODE);
Query OK, 0 rows affected (0.028 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Add a foreign key constraint to the ASSIGNMENT table linking CRS_CODE to the COURSE table

ALTER TABLE ASSIGNMENT

ADD CONSTRAINT FK_ASSIGNMENT_COURSE

FOREIGN KEY (CRS_CODE) REFERENCES COURSE(CRS_CODE);

-- Ensures that the CRS_CODE in the ASSIGNMENT table must exist in the COURSE table

Output

```
MariaDB [Online_Course]> -- Ensures that the CRS_CODE in the ENROLLMENT table must exist in the COURSE table
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Add a foreign key constraint to the ASSIGNMENT table linking CRS_CODE to the COURSE table
MariaDB [Online_Course]> ALTER TABLE ASSIGNMENT
    -> ADD CONSTRAINT FK_ASSIGNMENT_COURSE
    -> FOREIGN KEY (CRS_CODE) REFERENCES COURSE(CRS_CODE);
Query OK, 0 rows affected (0.028 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Add a foreign key constraint to the SUBMISSION table linking STUDENT_ID to the STUDENT table

ALTER TABLE SUBMISSION

ADD CONSTRAINT FK_SUBMISSION_STUDENT

FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT(STUDENT_ID);

-- Ensures that the STUDENT_ID in the SUBMISSION table must exist in the STUDENT table

Output

```
MariaDB [Online_Course]> -- Ensures that the CRS_CODE in the ASSIGNMENT table must exist in the COURSE table
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Add a foreign key constraint to the SUBMISSION table linking STUDENT_ID to the STUDENT table
MariaDB [Online_Course]> ALTER TABLE SUBMISSION
    -> ADD CONSTRAINT FK_SUBMISSION_STUDENT
    -> FOREIGN KEY (STUDENT_ID) REFERENCES STUDENT(STUDENT_ID);
Query OK, 0 rows affected (0.028 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Add a foreign key constraint to the SUBMISSION table linking ASSIGNMENT_ID to the ASSIGNMENT table

ALTER TABLE SUBMISSION

ADD CONSTRAINT FK_SUBMISSION_ASSIGNMENT

FOREIGN KEY (ASSIGNMENT_ID) REFERENCES ASSIGNMENT(ASSIGN_ID);

-- Ensures that the ASSIGNMENT_ID in the SUBMISSION table must exist in the ASSIGNMENT table

Output

```
MariaDB [Online_Course]> -- Ensures that the STUDENT_ID in the SUBMISSION table must exist in the STUDENT table
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Add a foreign key constraint to the SUBMISSION table linking ASSIGNMENT_ID to the ASSIGNMENT table
MariaDB [Online_Course]> ALTER TABLE SUBMISSION
    -> ADD CONSTRAINT FK_SUBMISSION_ASSIGNMENT
    -> FOREIGN KEY (ASSIGNMENT_ID) REFERENCES ASSIGNMENT(ASSIGN_ID);
Query OK, 0 rows affected (0.026 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

-- Add a foreign key constraint to the LEARNING_MATERIAL table linking CRS_CODE to the COURSE table

ALTER TABLE LEARNING_MATERIAL

ADD CONSTRAINT FK_MATERIAL_COURSE

FOREIGN KEY (CRS_CODE) REFERENCES COURSE(CRS_CODE);

-- Ensures that the CRS_CODE in the LEARNING_MATERIAL table must exist in the COURSE table

Output

```
MariaDB [Online_Course]> -- Ensures that the ASSIGNMENT_ID in the SUBMISSION table must exist in the ASSIGNMENT table
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Add a foreign key constraint to the LEARNING_MATERIAL table linking CRS_CODE to the COURSE table
MariaDB [Online_Course]> ALTER TABLE LEARNING_MATERIAL
    -> ADD CONSTRAINT FK_MATERIAL_COURSE
    -> FOREIGN KEY (CRS_CODE) REFERENCES COURSE(CRS_CODE);
Query OK, 0 rows affected (0.033 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

The constraints preserve that the records don't get disconnected in case of no key relations and make a sound and preserved integrity of the database.

3. <u>Data Insertion</u>

When the tables are then created, and example data is inserted for a real case scenario. For example:

- Students: 15 students with proper information duly filled with several such information such as name and email address, fill in the database.

-- Insert student records into the STUDENT table

INSERT INTO STUDENT VALUES

(1, 'Mohammed', 'Ali', 'mohammed1@mmu.edu.my', '0501234567', '2002-05-15', 'M', 'Riyadh'), -- Student 1

(2, 'Ahmed', 'Saleh', 'ahmed2@mmu.edu.my', '0502345678', '2001-03-10', 'M', 'Jeddah'), -- Student 2

(3, 'Fatima', 'Sharif', 'fatima3@mmu.edu.my', '0503456789', '2003-07-20', 'F', 'Mecca'), -- Student 3

(4, 'Khalid', 'Saeed', 'khalid4@mmu.edu.my', '0504567890', '2000-01-01', 'M', 'Dammam'), -- Student 4

(5, 'Noor', 'Hashimi', 'noor5@mmu.edu.my', '0505678901', '2002-12-31', 'F', 'Medina'), -- Student 5

(6, 'Abdullah', 'Ghamdi', 'abdullah6@mmu.edu.my', '0506789012', '1999-09-15', 'M', 'Riyadh'), -- Student 6

(7, 'Hind', 'Otaibi', 'hind7@mmu.edu.my', '0507890123', '2004-11-25', 'F', 'Jeddah'), -- Student 7

(8, 'Badr', 'Dosari', 'badr8@mmu.edu.my', '0508901234', '2001-08-14', 'M', 'Mecca'), -- Student 8

(9, 'Layla', 'Shahrani', 'layla9@mmu.edu.my', '0509012345', '2003-05-05', 'F', 'Dammam'), -- Student 9

(10, 'Saud', 'Zaid', 'saud10@mmu.edu.my', '0511234567', '2000-06-30', 'M', 'Riyadh'), -- Student 10

(11, 'Mona', 'Khalid', 'mona11@mmu.edu.my', '0512345678', '2002-03-22', 'F', 'Jeddah'), -- Student 11

(12, 'Abdulrahman', 'Tamimi', 'abrahman12@mmu.edu.my', '0513456789', '2001-02-28', 'M', 'Mecca'), -- Student 12

(13, 'Noura', 'Mutairi', 'nora13@mmu.edu.my', '0514567890', '1999-07-17', 'F', 'Dammam'), -- Student 13

(14, 'Salem', 'Qahtani', 'salem14@mmu.edu.my', '0515678901', '2004-10-13', 'M', 'Medina'), -- Student 14

(15, 'Jamila', 'Anzi', 'jamila15@mmu.edu.my', '0516789012', '2003-01-05', 'F', 'Jeddah'); -- Student 15

Output

```
MariaDB [Online_Course]> -- Ensures that the CRS_CODE in the LEARNING_MATERIAL table must exist in the COURSE table
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Insert student records into the STUDENT table
MariaDB [Online_Course]> INSERT INTO STUDENT VALUES
    -> (1, 'Mohammed', 'Ali', 'mohammed1@mmu.edu.my', '0501234567', '2002-05-15', 'M', 'Riyadh'),       -- Student 1
    -> (2, 'Ahmed', 'Saleh', 'ahmed2@mmu.edu.my', '0502345678', '2001-03-10', 'M', 'Jeddah'),          -- Student 2
    -> (3, 'Fatima', 'Sharif', 'fatima3@mmu.edu.my', '0503456789', '2003-07-20', 'F', 'Mecca'),        -- Student 3
    -> (4, 'Khalid', 'Saeed', 'khalid4@mmu.edu.my', '0504567890', '2000-01-01', 'M', 'Dammam'),        -- Student 4
    -> (5, 'Noor', 'Hashimi', 'noor5@mmu.edu.my', '0505678901', '2002-12-31', 'F', 'Medina'),          -- Student 5
    -> (6, 'Abdullah', 'Ghamdi', 'abdullah6@mmu.edu.my', '0506789012', '1999-09-15', 'M', 'Riyadh'),   -- Student 6
    -> (7, 'Hind', 'Otaibi', 'hind7@mmu.edu.my', '0507890123', '2004-11-25', 'F', 'Jeddah'),           -- Student 7
    -> (8, 'Badr', 'Dosari', 'badr8@mmu.edu.my', '0508901234', '2001-08-14', 'M', 'Mecca'),            -- Student 8
    -> (9, 'Layla', 'Shahrani', 'layla9@mmu.edu.my', '0509012345', '2003-05-05', 'F', 'Dammam'),       -- Student 9
    -> (10, 'Saud', 'Zaid', 'saud10@mmu.edu.my', '0511234567', '2000-06-30', 'M', 'Riyadh'),           -- Student 10
    -> (11, 'Mona', 'Khalid', 'mona11@mmu.edu.my', '0512345678', '2002-03-22', 'F', 'Jeddah'),         -- Student 11
    -> (12, 'Abdulrahman', 'Tamimi', 'abrahman12@mmu.edu.my', '0513456789', '2001-02-28', 'M', 'Mecca'), -- Student 12
    -> (13, 'Noura', 'Mutairi', 'nora13@mmu.edu.my', '0514567890', '1999-07-17', 'F', 'Dammam'),       -- Student 13
    -> (14, 'Salem', 'Qahtani', 'salem14@mmu.edu.my', '0515678901', '2004-10-13', 'M', 'Medina'),      -- Student 14
    -> (15, 'Jamila', 'Anzi', 'jamila15@mmu.edu.my', '0516789012', '2003-01-05', 'F', 'Jeddah');       -- Student 15
Query OK, 15 rows affected (0.004 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

- Instructors: Four qualified istructors fill in the database.

-- Insert instructor records into the INSTRUCTOR table

INSERT INTO INSTRUCTOR VALUES

(1, 'Dr. Hassan Al-Sayed', 'hassan@mmu.edu.my', '0502233445', 'Advanced Mathematics'), -- Instructor 1

(2, 'Prof. Layla Mahmoud', 'layla@mmu.edu.my', '0503344556', 'Computer Science'), -- Instructor 2

(3, 'Dr. Faisal Al-Harbi', 'faisal@mmu.edu.my', '0504455667', 'Physics'), -- Instructor 3

(4, 'Dr. Mona Al-Qassem', 'mona@mmu.edu.my', '0505566778', 'Economics');

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Insert instructor records into the INSTRUCTOR table
MariaDB [Online_Course]> INSERT INTO INSTRUCTOR VALUES
    -> (1, 'Dr. Hassan Al-Sayed', 'hassan@mmu.edu.my', '0502233445', 'Advanced Mathematics'),          -- Instructor 1
    -> (2, 'Prof. Layla Mahmoud', 'layla@mmu.edu.my', '0503344556', 'Computer Science'),               -- Instructor 2
    -> (3, 'Dr. Faisal Al-Harbi', 'faisal@mmu.edu.my', '0504455667', 'Physics'),                       -- Instructor 3
    -> (4, 'Dr. Mona Al-Qassem', 'mona@mmu.edu.my', '0505566778', 'Economics');                        -- Instructor 4
Query OK, 4 rows affected (0.002 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

- Courses: There must have at least 5 courses, and each will refer to an instructor.

-- Insert course records into the COURSE table

INSERT INTO COURSE VALUES

(101, 'Calculus I', 'Introduction to differential and integral calculus.', '08:00 AM - 10:00 AM', '2025-01-01', '2025-06-01', 'http://lecture1.MMU.edu.my', 1500.00, 1), -- Course 101

(102, 'Introduction to Programming', 'Basics of programming using Python.', '10:30 AM - 12:30 PM', '2025-01-01', '2025-06-01', 'http://lecture2.MMU.edu.my', 1800.00, 2), -- Course 102

(103, 'Physics I', 'Fundamentals of classical mechanics.', '01:00 PM - 03:00 PM', '2025-01-01', '2025-06-01', 'http://lecture3.MMU.edu.my', 1700.00, 3), -- Course 103

(104, 'Macroeconomics', 'Overview of macroeconomic principles.', '03:30 PM - 05:30 PM', '2025-01-01', '2025-06-01', 'http://lecture4.MMU.edu.my', 1600.00, 4), -- Course 104

(105, 'Data Structures', 'Introduction to data structures in programming.', '06:00 PM - 08:00 PM', '2025-01-01', '2025-06-01', 'http://lecture5.MMU.edu.my', 2000.00, 2); -- Course 105

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Insert course records into the COURSE table
MariaDB [Online_Course]> INSERT INTO COURSE VALUES
    -> (101, 'Calculus I', 'Introduction to differential and integral calculus.', '08:00 AM - 10:00 AM', '2025-01-01', '2025-06-01', 'http://lecture1.MMU.edu.my', 1500.00, 1), -- Course 101
    -> (102, 'Introduction to Programming', 'Basics of programming using Python.', '10:30 AM - 12:30 PM', '2025-01-01', '2025-06-01', 'http://lecture2.MMU.edu.my', 1800.00, 2), -- Course 102
    -> (103, 'Physics I', 'Fundamentals of classical mechanics.', '01:00 PM - 03:00 PM', '2025-01-01', '2025-06-01', 'http://lecture3.MMU.edu.my', 1700.00, 3), -- Course 103
    -> (104, 'Macroeconomics', 'Overview of macroeconomic principles.', '03:30 PM - 05:30 PM', '2025-01-01', '2025-06-01', 'http://lecture4.MMU.edu.my', 1600.00, 4), -- Course 104
    -> (105, 'Data Structures', 'Introduction to data structures in programming.', '06:00 PM - 08:00 PM', '2025-01-01', '2025-06-01', 'http://lecture5.MMU.edu.my', 2000.00, 2); -- Course 105
Query OK, 5 rows affected (0.004 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

- Enrollments: Students are registered to courses with specific date of enrollments and payment status.

-- Insert enrollment records into the ENROLLMENT table

INSERT INTO ENROLLMENT VALUES

(1, 101, '2025-01-10', 'In Progress', NULL, 'Paid'), -- Student 1 enrolled in Course 101

(2, 102, '2025-01-12', 'In Progress', NULL, 'Paid'), -- Student 2 enrolled in Course 102

(3, 103, '2025-01-15', 'In Progress', NULL, 'Paid'), -- Student 3 enrolled in Course 103

(4, 104, '2025-01-20', 'In Progress', NULL, 'Paid'), -- Student 4 enrolled in Course 104

(5, 105, '2025-01-25', 'In Progress', NULL, 'Paid'); -- Student 5 enrolled in Course 105

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Insert enrollment records into the ENROLLMENT table
MariaDB [Online_Course]> INSERT INTO ENROLLMENT VALUES
    -> (1, 101, '2025-01-10', 'In Progress', NULL, 'Paid'),  -- Student 1 enrolled in Course 101
    -> (2, 102, '2025-01-12', 'In Progress', NULL, 'Paid'),  -- Student 2 enrolled in Course 102
    -> (3, 103, '2025-01-15', 'In Progress', NULL, 'Paid'),  -- Student 3 enrolled in Course 103
    -> (4, 104, '2025-01-20', 'In Progress', NULL, 'Paid'),  -- Student 4 enrolled in Course 104
    -> (5, 105, '2025-01-25', 'In Progress', NULL, 'Paid');  -- Student 5 enrolled in Course 105
Query OK, 5 rows affected (0.003 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

- Assignments: Assignments for each course with specific due date and description are inserted.

-- Insert assignment records into the ASSIGNMENT table

INSERT INTO ASSIGNMENT VALUES

(1, 'Assignment 1', 'Basic Calculus Problems', '2025-02-01', 101), -- Assignment for Course 101

(2, 'Assignment 2', 'Python Programming Basics', '2025-02-05', 102), -- Assignment for Course 102

(3, 'Assignment 3', 'Classical Mechanics Exercise', '2025-02-10', 103), -- Assignment for Course 103

(4, 'Assignment 4', 'Economic Principles Case Study', '2025-02-15', 104), -- Assignment for Course 104

(5, 'Assignment 5', 'Linked List Implementation', '2025-02-20', 105); -- Assignment for Course 105

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Insert assignment records into the ASSIGNMENT table
MariaDB [Online_Course]> INSERT INTO ASSIGNMENT VALUES
    -> (1, 'Assignment 1', 'Basic Calculus Problems', '2025-02-01', 101),  -- Assignment for Course 101
    -> (2, 'Assignment 2', 'Python Programming Basics', '2025-02-05', 102), -- Assignment for Course 102
    -> (3, 'Assignment 3', 'Classical Mechanics Exercise', '2025-02-10', 103), -- Assignment for Course 103
    -> (4, 'Assignment 4', 'Economic Principles Case Study', '2025-02-15', 104), -- Assignment for Course 104
    -> (5, 'Assignment 5', 'Linked List Implementation', '2025-02-20', 105); -- Assignment for Course 105
Query OK, 5 rows affected (0.003 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

- Learning Materials: Materials, such as videos or pdfs, for each course are inserted.

-- Insert learning material records into the LEARNING_MATERIAL table
INSERT INTO LEARNING_MATERIAL VALUES
(1, 'Calculus Lecture Notes', 'PDF', '2025-01-05', 101, 'Public'), -- Learning material for Course 101
(2, 'Python Code Examples', 'ZIP', '2025-01-10', 102, 'Restricted'), -- Learning material for Course 102
(3, 'Mechanics Problem Sets', 'PDF', '2025-01-15', 103, 'Public'), -- Learning material for Course 103
(4, 'Economics Charts', 'PPT', '2025-01-20', 104, 'Public'), -- Learning material for Course 104
(5, 'Data Structures Diagrams', 'PDF', '2025-01-25', 105, 'Restricted'); -- Learning material for Course 105

Output

```
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Insert learning material records into the LEARNING_MATERIAL table
MariaDB [Online_Course]> INSERT INTO LEARNING_MATERIAL VALUES
    -> (1, 'Calculus Lecture Notes', 'PDF', '2025-01-05', 101, 'Public'),      -- Learning material for Course 101
    -> (2, 'Python Code Examples', 'ZIP', '2025-01-10', 102, 'Restricted'),    -- Learning material for Course 102
    -> (3, 'Mechanics Problem Sets', 'PDF', '2025-01-15', 103, 'Public'),      -- Learning material for Course 103
    -> (4, 'Economics Charts', 'PPT', '2025-01-20', 104, 'Public'),            -- Learning material for Course 104
    -> (5, 'Data Structures Diagrams', 'PDF', '2025-01-25', 105, 'Restricted'); -- Learning material for Course 105
Query OK, 5 rows affected (0.006 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

The data is inserted in order to test the system and make a confirm that everything works perfectly fine.

4: Trigger:

Triggers are specially designed SQL routines, which execute automatically when a certain event occurs (e.g., inserting or updating information). They make sure that integrity and business rules are preserved and incorrect information cannot enter the system. Here's what each trigger does:

Trigger 1: Prevent duplicate enrollment:

Prevent Duplicate Enrollment: Won't permit a student to enroll for a course for a second (duplicate) occasion. In case a duplicate enrollment is entered, a message for an error is thrown.

```
-- Change the delimiter to $$ to allow the creation of the trigger
DELIMITER $$

-- Create a trigger named `trg_prevent_duplicate_enrollment`
-- This trigger fires BEFORE an INSERT operation on the ENROLLMENT table
CREATE TRIGGER trg_prevent_duplicate_enrollment
BEFORE INSERT ON ENROLLMENT
FOR EACH ROW
BEGIN
   -- Declare a variable to store the count of existing enrollments for the student and course
   DECLARE count_enrollments INT;

   -- Check if the student is already enrolled in the course
   SELECT COUNT(*) INTO count_enrollments
   FROM ENROLLMENT
   WHERE STUDENT_ID = NEW.STUDENT_ID AND CRS_CODE = NEW.CRS_CODE;

   -- If the student is already enrolled, raise an error
   IF count_enrollments > 0 THEN
      SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
      SET MESSAGE_TEXT = 'Duplicate enrollment detected.'; -- Error message
   END IF;
END$$

-- Reset the delimiter back to the default semicolon
```

DELIMITER ;

Output

```
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the trigger
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a trigger named `trg_prevent_duplicate_enrollment`
MariaDB [Online_Course]> -- This trigger fires BEFORE an INSERT operation on the ENROLLMENT table
MariaDB [Online_Course]> CREATE TRIGGER trg_prevent_duplicate_enrollment
    -> BEFORE INSERT ON ENROLLMENT
    -> FOR EACH ROW
    -> BEGIN
    ->     -- Declare a variable to store the count of existing enrollments for the student and course
    ->     DECLARE count_enrollments INT;
    ->
    ->     -- Check if the student is already enrolled in the course
    ->     SELECT COUNT(*) INTO count_enrollments
    ->     FROM ENROLLMENT
    ->     WHERE STUDENT_ID = NEW.STUDENT_ID AND CRS_CODE = NEW.CRS_CODE;
    ->
    ->     -- If the student is already enrolled, raise an error
    ->     IF count_enrollments > 0 THEN
    ->         SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
    ->         SET MESSAGE_TEXT = 'Duplicate enrollment detected.'; -- Error message
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.009 sec)
```

Test the trigger with this student:

STUDENT_ID	CRS_CODE	ENROL_DATE	COMPLETION_STATE	GRADE	PAYMENT
1	101	2023-10-15	In Progress	NULL	Paid
2	102	2025-01-12	In Progress	NULL	Paid
3	102	2025-01-25	Completed	A	Paid
3	103	2025-01-15	In Progress	NULL	Paid
4	104	2025-01-20	In Progress	NULL	Paid
5	105	2025-01-25	In Progress	NULL	Paid

```
MariaDB [Online_course]> INSERT INTO ENROLLMENT (STUDENT_ID, CRS_CODE, ENROL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
    -> VALUES (1, 101, '2023-10-15', 'In Progress', NULL, 'Paid');
ERROR 1644 (45000): Duplicate enrollment detected.
```

Trigger 2. Prevent Invalid Enrollment Dates

This trigger ensures that the enrollment date (ENROL_DATE) is not in the future:

Prevent Invalid Enrollment Dates: Won't permit an enrollment date in the future to be entered. In case a future date is entered, a message for an error is thrown.

-- Change the delimiter to $$ to allow the creation of the trigger

```
DELIMITER $$

-- Create a trigger named `trg_prevent_future_enrollment`
-- This trigger fires BEFORE an INSERT operation on the ENROLLMENT table
CREATE TRIGGER trg_prevent_future_enrollment
BEFORE INSERT ON ENROLLMENT
FOR EACH ROW
BEGIN
   -- Check if the ENROL_DATE in the new record is in the future
   IF NEW.ENROL_DATE > CURDATE() THEN
      -- Raise an error if the enrollment date is in the future
      SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
      SET MESSAGE_TEXT = 'Enrollment date cannot be in the future.'; -- Error message
   END IF;
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the trigger
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a trigger named `trg_prevent_future_enrollment`
MariaDB [Online_Course]> -- This trigger fires BEFORE an INSERT operation on the ENROLLMENT table
MariaDB [Online_Course]> CREATE TRIGGER trg_prevent_future_enrollment
    -> BEFORE INSERT ON ENROLLMENT
    -> FOR EACH ROW
    -> BEGIN
    ->     -- Check if the ENROL_DATE in the new record is in the future
    ->     IF NEW.ENROL_DATE > CURDATE() THEN
    ->         -- Raise an error if the enrollment date is in the future
    ->         SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
    ->         SET MESSAGE_TEXT = 'Enrollment date cannot be in the future.'; -- Error message
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.009 sec)
```

Test the trigger

```
MariaDB [Online_course]> INSERT INTO ENROLLMENT (STUDENT_ID, CRS_CODE, ENROL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
    -> VALUES (6, 101, '2025-03-01', 'In Progress', NULL, 'Paid');
ERROR 1644 (45000): Enrollment date cannot be in the future.
```

Trigger 3: Prevent Assignment Submission After Due Date:

Prevent Late Submissions: Won't permit students to submit an assignment after its due date. In case a submission is attempted, an error message is thrown.

```
-- Change the delimiter to $$ to allow the creation of the trigger
DELIMITER $$

-- Create a trigger named `trg_prevent_late_submission`
-- This trigger fires BEFORE an INSERT operation on the SUBMISSION table
CREATE TRIGGER trg_prevent_late_submission
BEFORE INSERT ON SUBMISSION
FOR EACH ROW
BEGIN
   -- Declare a variable to store the due date of the assignment
   DECLARE due_date DATE;

   -- Retrieve the due date for the assignment being submitted
   SELECT ASSIGN_DUE_DATE INTO due_date
   FROM ASSIGNMENT
   WHERE ASSIGN_ID = NEW.ASSIGNMENT_ID;

   -- Check if the submission date is after the due date
   IF NEW.SUBM_DATE > due_date THEN
      -- Raise an error if the submission is late
      SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
      SET MESSAGE_TEXT = 'Assignment submission is past the due date.'; -- Error message
   END IF;
END$$
```

-- Reset the delimiter back to the default semicolon

DELIMITER ;

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the trigger
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a trigger named `trg_prevent_late_submission`
MariaDB [Online_Course]> -- This trigger fires BEFORE an INSERT operation on the SUBMISSION table
MariaDB [Online_Course]> CREATE TRIGGER trg_prevent_late_submission
    -> BEFORE INSERT ON SUBMISSION
    -> FOR EACH ROW
    -> BEGIN
    ->     -- Declare a variable to store the due date of the assignment
    ->     DECLARE due_date DATE;
    ->
    ->     -- Retrieve the due date for the assignment being submitted
    ->     SELECT ASSIGN_DUE_DATE INTO due_date
    ->     FROM ASSIGNMENT
    ->     WHERE ASSIGN_ID = NEW.ASSIGNMENT_ID;
    ->
    ->     -- Check if the submission date is after the due date
    ->     IF NEW.SUBM_DATE > due_date THEN
    ->         -- Raise an error if the submission is late
    ->         SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
    ->         SET MESSAGE_TEXT = 'Assignment submission is past the due date.'; -- Error message
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.010 sec)
```

Test the trigger

```
MariaDB [Online_course]> INSERT INTO SUBMISSION (STUDENT_ID, ASSIGNMENT_ID, SUBM_DATE, SUBM_GRADE)
    -> VALUES (2, 2, '2025-02-06', 'A');
ERROR 1644 (45000): Assignment submission is past the due date.
MariaDB [Online_course]>
```

The triggers maintain the integrity of the system and enforce rules automatically.

5: <u>Function</u>

Functions are repeatable blocks of code, which accomplish a specific purpose and spit out a value. Here's what each function does:

Function 1: Calculate average grade for a course:

Converts letter values (A, B, C, etc.) into numerical values and calculates an average mark for a course.

```
-- Change the delimiter to $$ to allow the creation of the function
DELIMITER $$

-- Create a function named `avg_grade_course`
-- This function calculates the average grade for a given course code
CREATE FUNCTION avg_grade_course(course_code INT) RETURNS FLOAT
BEGIN
   -- Declare a variable to store the calculated average grade
   DECLARE avg_grade FLOAT;

   -- Calculate the average grade for the course
   SELECT AVG(
      CASE
         WHEN SUBM_GRADE = 'A' THEN 4.0 -- Map 'A' to 4.0
         WHEN SUBM_GRADE = 'B' THEN 3.0 -- Map 'B' to 3.0
         WHEN SUBM_GRADE = 'C' THEN 2.0 -- Map 'C' to 2.0
         WHEN SUBM_GRADE = 'D' THEN 1.0 -- Map 'D' to 1.0
         WHEN SUBM_GRADE = 'F' THEN 0.0 -- Map 'F' to 0.0
         ELSE NULL -- Ignore invalid or NULL grades
      END
   ) INTO avg_grade
   FROM SUBMISSION S
   JOIN ASSIGNMENT A ON S.ASSIGNMENT_ID = A.ASSIGN_ID
   WHERE A.CRS_CODE = course_code; -- Filter by the provided course code

   -- Return the average grade, or 0 if no grades are found (to handle NULL cases)
   RETURN IFNULL(avg_grade, 0);
```

END$$

-- Reset the delimiter back to the default semicolon

DELIMITER ;

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the function
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a function named `avg_grade_course`
MariaDB [Online_Course]> -- This function calculates the average grade for a given course code
MariaDB [Online_Course]> CREATE FUNCTION avg_grade_course(course_code INT) RETURNS FLOAT
    -> BEGIN
    ->     -- Declare a variable to store the calculated average grade
    ->     DECLARE avg_grade FLOAT;
    ->
    ->     -- Calculate the average grade for the course
    ->     SELECT AVG(
    ->         CASE
    ->             WHEN SUBM_GRADE = 'A' THEN 4.0 -- Map 'A' to 4.0
    ->             WHEN SUBM_GRADE = 'B' THEN 3.0 -- Map 'B' to 3.0
    ->             WHEN SUBM_GRADE = 'C' THEN 2.0 -- Map 'C' to 2.0
    ->             WHEN SUBM_GRADE = 'D' THEN 1.0 -- Map 'D' to 1.0
    ->             WHEN SUBM_GRADE = 'F' THEN 0.0 -- Map 'F' to 0.0
    ->             ELSE NULL -- Ignore invalid or NULL grades
    ->         END
    ->     ) INTO avg_grade
    ->     FROM SUBMISSION S
    ->     JOIN ASSIGNMENT A ON S.ASSIGNMENT_ID = A.ASSIGN_ID
    ->     WHERE A.CRS_CODE = course_code; -- Filter by the provided course code
    ->
    ->     -- Return the average grade, or 0 if no grades are found (to handle NULL cases)
    ->     RETURN IFNULL(avg_grade, 0);
    -> END$$
Query OK, 0 rows affected (0.004 sec)
```

Call the function

```
MariaDB [Online_Course]> SELECT avg_grade_course(101);
+------------------------+
| avg_grade_course(101) |
+------------------------+
|                      4 |
+------------------------+
1 row in set (0.001 sec)
```

Function 2: Get total payments made by a student:

Get Total Payments Made by a Student: Sums up fee for each course a student is studying for.

```
DELIMITER $$

-- Create a function named `total_payments` that takes a `student_id` as input
-- and returns the total amount of fees paid by the student as a FLOAT value.
CREATE FUNCTION total_payments(student_id INT) RETURNS FLOAT
BEGIN
   -- Declare a variable `total` to store the sum of the course fees.
   DECLARE total FLOAT;

   -- Calculate the total fees paid by the student by summing up the `CRS_FEES`
   -- from the `COURSE` table for all courses where the student has made a payment.
   -- The `ENROLLMENT` table is joined with the `COURSE` table using the `CRS_CODE`
   -- to get the course fees. The condition `E.PAYMENT = 'Paid'` ensures that only
   -- paid enrollments are considered.
   SELECT SUM(C.CRS_FEES) INTO total
   FROM ENROLLMENT E
   JOIN COURSE C ON E.CRS_CODE = C.CRS_CODE
   WHERE E.STUDENT_ID = student_id AND E.PAYMENT = 'Paid';

   -- Return the total fees paid by the student. If no payments were found,
   -- return 0 using the `IFNULL` function to handle cases where `total` is NULL.
   RETURN IFNULL(total, 0);
END$$
```

DELIMITER ;

Output

```
MariaDB [online_course]> DELIMITER $$
MariaDB [online_course]>
MariaDB [online_course]> -- Create a function named `total_payments` that takes a `student_id` as input
MariaDB [online_course]> -- and returns the total amount of fees paid by the student as a FLOAT value.
MariaDB [online_course]> CREATE FUNCTION total_payments(student_id INT) RETURNS FLOAT
    -> BEGIN
    ->     -- Declare a variable `total` to store the sum of the course fees.
    ->     DECLARE total FLOAT;
    ->
    ->     -- Calculate the total fees paid by the student by summing up the `CRS_FEES`
    ->     -- from the `COURSE` table for all courses where the student has made a payment.
    ->     -- The `ENROLLMENT` table is joined with the `COURSE` table using the `CRS_CODE`
    ->     -- to get the course fees. The condition `E.PAYMENT = 'Paid'` ensures that only
    ->     -- paid enrollments are considered.
    ->     SELECT SUM(C.CRS_FEES) INTO total
    ->     FROM ENROLLMENT E
    ->     JOIN COURSE C ON E.CRS_CODE = C.CRS_CODE
    ->     WHERE E.STUDENT_ID = student_id AND E.PAYMENT = 'Paid';
    ->
    ->     -- Return the total fees paid by the student. If no payments were found,
    ->     -- return 0 using the `IFNULL` function to handle cases where `total` is NULL.
    ->     RETURN IFNULL(total, 0);
    -> END$$
Query OK, 0 rows affected (0.004 sec)
```

Call the function

```
MariaDB [Online_Course]> SELECT total_payments(1);
+-------------------+
| total_payments(1) |
+-------------------+
|              1500 |
+-------------------+
1 row in set (0.002 sec)

MariaDB [Online_Course]> |
```

Function 3: Count total students in a course:

Returns a value for students in a particular course.

-- Change the delimiter to $$ to allow the creation of the function

DELIMITER $$

```
-- Create a function named `count_students_in_course`
-- This function calculates the total number of students enrolled in a specific course
CREATE FUNCTION count_students_in_course(course_code INT) RETURNS INT
BEGIN
    -- Declare a variable to store the total number of students
    DECLARE total_students INT;

    -- Count the number of students enrolled in the course
    SELECT COUNT(*) INTO total_students
    FROM ENROLLMENT
    WHERE CRS_CODE = course_code; -- Filter by the provided course code

    -- Return the total number of students enrolled in the course
    RETURN total_students;
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the function
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a function named `count_students_in_course`
MariaDB [Online_Course]> -- This function calculates the total number of students enrolled in a specific course
MariaDB [Online_Course]> CREATE FUNCTION count_students_in_course(course_code INT) RETURNS INT
    -> BEGIN
    ->     -- Declare a variable to store the total number of students
    ->     DECLARE total_students INT;
    ->
    ->     -- Count the number of students enrolled in the course
    ->     SELECT COUNT(*) INTO total_students
    ->     FROM ENROLLMENT
    ->     WHERE CRS_CODE = course_code; -- Filter by the provided course code
    ->
    ->     -- Return the total number of students enrolled in the course
    ->     RETURN total_students;
    -> END$$
Query OK, 0 rows affected (0.025 sec)
```

Call function

```
MariaDB [Online_Course]> SELECT count_students_in_course(101);
+-------------------------------+
| count_students_in_course(101) |
+-------------------------------+
|                             1 |
+-------------------------------+
1 row in set (0.001 sec)
```

Function 4: Check if material is accessible by permission:

Returns a value for a specific access permission a studying material holds (e.g., "Public" or "Restricted").

1 represents TRUE.

0 represents FALSE.

```
-- Change the delimiter to $$ to allow the creation of the function
DELIMITER $$

-- Create a function named `check_material_permission`
-- This function checks if a learning material has a specific access permission
CREATE FUNCTION check_material_permission(material_id INT, permission NVARCHAR(50)) RETURNS BOOLEAN
BEGIN
   -- Declare a variable to store the permission check result
   DECLARE has_permission BOOLEAN;

   -- Check if the learning material has the specified permission
   SELECT CASE
      WHEN ACCESS_PERMISSION = permission THEN TRUE -- Return TRUE if permissions match
      ELSE FALSE -- Return FALSE if permissions do not match
   END INTO has_permission
```

```
    FROM LEARNING_MATERIAL
    WHERE MATER_ID = material_id; -- Filter by the provided material ID

    -- Return the result of the permission check
    RETURN has_permission;
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the function
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a function named `check_material_permission`
MariaDB [Online_Course]> -- This function checks if a learning material has a specific access permission
MariaDB [Online_Course]> CREATE FUNCTION check_material_permission(material_id INT, permission NVARCHAR(50)) RETURNS BOOLEAN
    -> BEGIN
    ->     -- Declare a variable to store the permission check result
    ->     DECLARE has_permission BOOLEAN;
    ->
    ->     -- Check if the learning material has the specified permission
    ->     SELECT CASE
    ->         WHEN ACCESS_PERMISSION = permission THEN TRUE -- Return TRUE if permissions match
    ->         ELSE FALSE -- Return FALSE if permissions do not match
    ->     END INTO has_permission
    ->     FROM LEARNING_MATERIAL
    ->     WHERE MATER_ID = material_id; -- Filter by the provided material ID
    ->
    ->     -- Return the result of the permission check
    ->     RETURN has_permission;
    -> END$$
Query OK, 0 rows affected (0.025 sec)
```

Call the function

```
MariaDB [Online_Course]> SELECT check_material_permission(1, 'Public');
+----------------------------------------+
| check_material_permission(1, 'Public') |
+----------------------------------------+
|                                      1 |
+----------------------------------------+
1 row in set (0.001 sec)
```

Function 5: Calculate days left for assignment deadline:

Returns a value for days for an assignment's due date.

```
-- Change the delimiter to $$ to allow the creation of the function
DELIMITER $$

-- Create a function named `days_to_deadline`
-- This function calculates the number of days remaining until the deadline of an assignment
CREATE FUNCTION days_to_deadline(assign_id INT) RETURNS INT
BEGIN
   -- Declare a variable to store the number of days left
   DECLARE days_left INT;

   -- Calculate the number of days between the assignment due date and the current date
   SELECT DATEDIFF(ASSIGN_DUE_DATE, CURDATE()) INTO days_left
   FROM ASSIGNMENT
   WHERE ASSIGN_ID = assign_id -- Filter by the provided assignment ID
   LIMIT 1; -- Ensure only one row is returned

   -- Return the number of days left, or 0 if no assignment is found (to handle NULL cases)
   RETURN IFNULL(days_left, 0);
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the function
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a function named `days_to_deadline`
MariaDB [Online_Course]> -- This function calculates the number of days remaining until the deadline of an assignment
MariaDB [Online_Course]> CREATE FUNCTION days_to_deadline(assign_id INT) RETURNS INT
    -> BEGIN
    ->     -- Declare a variable to store the number of days left
    ->     DECLARE days_left INT;
    ->
    ->     -- Calculate the number of days between the assignment due date and the current date
    ->     SELECT DATEDIFF(ASSIGN_DUE_DATE, CURDATE()) INTO days_left
    ->     FROM ASSIGNMENT
    ->     WHERE ASSIGN_ID = assign_id -- Filter by the provided assignment ID
    ->     LIMIT 1; -- Ensure only one row is returned
    ->
    ->     -- Return the number of days left, or 0 if no assignment is found (to handle NULL cases)
    ->     RETURN IFNULL(days_left, 0);
    -> END$$
Query OK, 0 rows affected (0.006 sec)
```

Call the function

```
MariaDB [Online_Course]> SELECT days_to_deadline(1);
+----------------------+
| days_to_deadline(1) |
+----------------------+
|                    7 |
+----------------------+
1 row in set (0.001 sec)
```

The functions permit useful information retrieval with ease, without having to repeat complex queries again and again.

6: Procedure

Procedure 1: Enroll a student into a course

Pre-written SQL commands are what stored procedures are. They have the advantage of simplifying the execution of complex operations by using single commands. Procedures will do the following:

Enroll Student into Course: this procedure will enroll a student into a course with details to include completion status, grade, and payment status. It automatically sets the enrollment date as the current date.

```
-- Change the delimiter to $$ to allow the creation of the procedure
DELIMITER $$

-- Create a stored procedure named `enroll_student`
-- This procedure enrolls a student into a course with the provided details
CREATE PROCEDURE enroll_student(
   IN student_id INT,              -- Input parameter: Student ID
   IN course_code INT,             -- Input parameter: Course code
   IN completion_state NVARCHAR(50), -- Input parameter: Completion state (e.g., 'In Progress')
   IN grade NVARCHAR(2),           -- Input parameter: Grade (e.g., 'A', 'B', 'C')
   IN payment NVARCHAR(50)         -- Input parameter: Payment status (e.g., 'Paid', 'Pending')
)
BEGIN
   -- Insert a new record into the ENROLLMENT table
   INSERT INTO ENROLLMENT (STUDENT_ID, CRS_CODE, ENROL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
   VALUES (student_id, course_code, CURDATE(), completion_state, grade, payment);
   -- CURDATE() automatically sets the enrollment date to the current date
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the procedure
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a stored procedure named `enroll_student`
MariaDB [Online_Course]> -- This procedure enrolls a student into a course with the provided details
MariaDB [Online_Course]> CREATE PROCEDURE enroll_student(
    ->     IN student_id INT,                -- Input parameter: Student ID
    ->     IN course_code INT,               -- Input parameter: Course code
    ->     IN completion_state NVARCHAR(50), -- Input parameter: Completion state (e.g., 'In Progress')
    ->     IN grade NVARCHAR(2),             -- Input parameter: Grade (e.g., 'A', 'B', 'C')
    ->     IN payment NVARCHAR(50)           -- Input parameter: Payment status (e.g., 'Paid', 'Pending')
    -> )
    -> BEGIN
    ->     -- Insert a new record into the ENROLLMENT table
    ->     INSERT INTO ENROLLMENT (STUDENT_ID, CRS_CODE, ENROL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
    ->     VALUES (student_id, course_code, CURDATE(), completion_state, grade, payment);
    ->     -- CURDATE() automatically sets the enrollment date to the current date
    -> END$$
Query OK, 0 rows affected (0.025 sec)
```

Call the procedure

```
MariaDB [Online_course]> CALL enroll_student(3, 102, 'Completed', 'A', 'Paid');
Query OK, 1 row affected (0.005 sec)
```

```
MariaDB [Online_course]> select * from enrollment;
+------------+----------+------------+------------------+-------+---------+
| STUDENT_ID | CRS_CODE | ENROL_DATE | COMPLETION_STATE | GRADE | PAYMENT |
+------------+----------+------------+------------------+-------+---------+
|          1 |      101 | 2025-01-10 | In Progress      | NULL  | Paid    |
|          2 |      101 | 2025-01-25 | In Progress      | NULL  | Paid    |
|          2 |      102 | 2025-01-12 | In Progress      | NULL  | Paid    |
|          3 |      102 | 2025-01-25 | Completed        | A     | Paid    |
|          3 |      103 | 2025-01-15 | In Progress      | NULL  | Paid    |
|          4 |      104 | 2025-01-20 | In Progress      | NULL  | Paid    |
|          5 |      105 | 2025-01-25 | In Progress      | NULL  | Paid    |
+------------+----------+------------+------------------+-------+---------+
```

Procedure 2: Submit an assignment

Create Assignment: Allows a student to submit an assignment and ensures that no duplicate submissions are made. In case the student has already submitted the assignment, then an error is raised.

-- Change the delimiter to $$ to allow the creation of the procedure

DELIMITER $$

-- Create a stored procedure named `submit_assignment`

```
-- This procedure allows a student to submit an assignment, ensuring no duplicate submissions
CREATE PROCEDURE submit_assignment(
   IN student_id INT,      -- Input parameter: Student ID
   IN assign_id INT,       -- Input parameter: Assignment ID
   IN subm_date DATE,      -- Input parameter: Submission date
   IN grade NVARCHAR(2)    -- Input parameter: Grade for the submission
)
BEGIN
   -- Declare a variable to store the count of existing submissions for the student and assignment
   DECLARE submission_count INT;

   -- Check if the student has already submitted THIS assignment
   SELECT COUNT(*) INTO submission_count
   FROM SUBMISSION
   WHERE STUDENT_ID = student_id AND ASSIGNMENT_ID = assign_id;

   -- If the student has NOT submitted the assignment, insert the new submission
   IF submission_count = 0 THEN
      INSERT INTO SUBMISSION (STUDENT_ID, ASSIGNMENT_ID, SUBM_DATE, SUBM_GRADE)
      VALUES (student_id, assign_id, subm_date, grade);
   ELSE
      -- If the student has already submitted the assignment, raise an error
      SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
      SET MESSAGE_TEXT = 'Student has already submitted this assignment.'; -- Error message
   END IF;
END$$
```

-- Reset the delimiter back to the default semicolon

DELIMITER ;

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the procedure
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a stored procedure named `submit_assignment`
MariaDB [Online_Course]> -- This procedure allows a student to submit an assignment, ensuring no duplicate submissions
MariaDB [Online_Course]> CREATE PROCEDURE submit_assignment(
    ->     IN student_id INT,      -- Input parameter: Student ID
    ->     IN assign_id INT,       -- Input parameter: Assignment ID
    ->     IN subm_date DATE,      -- Input parameter: Submission date
    ->     IN grade NVARCHAR(2)    -- Input parameter: Grade for the submission
    -> )
    -> BEGIN
    ->     -- Declare a variable to store the count of existing submissions for the student and assignment
    ->     DECLARE submission_count INT;
    ->
    ->     -- Check if the student has already submitted THIS assignment
    ->     SELECT COUNT(*) INTO submission_count
    ->     FROM SUBMISSION
    ->     WHERE STUDENT_ID = student_id AND ASSIGNMENT_ID = assign_id;
    ->
    ->     -- If the student has NOT submitted the assignment, insert the new submission
    ->     IF submission_count = 0 THEN
    ->         INSERT INTO SUBMISSION (STUDENT_ID, ASSIGNMENT_ID, SUBM_DATE, SUBM_GRADE)
    ->         VALUES (student_id, assign_id, subm_date, grade);
    ->     ELSE
    ->         -- If the student has already submitted the assignment, raise an error
    ->         SIGNAL SQLSTATE '45000' -- SQLSTATE for a custom error
    ->         SET MESSAGE_TEXT = 'Student has already submitted this assignment.'; -- Error message
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.025 sec)
```

Call the procedure

```
MariaDB [Online_course]> CALL submit_assignment(1, 1, '2025-02-01', 'A');
Query OK, 2 rows affected (0.004 sec)
```

```
MariaDB [Online_course]> select * from SUBMISSION;
+------------+---------------+------------+------------+
| STUDENT_ID | ASSIGNMENT_ID | SUBM_DATE  | SUBM_GRADE |
+------------+---------------+------------+------------+
|          1 |             1 | 2025-02-01 | A          |
+------------+---------------+------------+------------+
1 row in set (0.001 sec)
```

Procedure 3: Update course fees

Updates the fees for a given course.

```
-- Change the delimiter to $$ to allow the creation of the procedure
DELIMITER $$

-- Create a stored procedure named `update_course_fees`
-- This procedure updates the fees for a specific course
CREATE PROCEDURE update_course_fees(
    IN course_code INT,  -- Input parameter: Course code to update
    IN new_fees FLOAT    -- Input parameter: New fees to set for the course
)
BEGIN
    -- Update the fees for the specified course
    UPDATE COURSE
    SET CRS_FEES = new_fees
    WHERE CRS_CODE = course_code; -- Filter by the provided course code
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the procedure
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a stored procedure named `update_course_fees`
MariaDB [Online_Course]> -- This procedure updates the fees for a specific course
MariaDB [Online_Course]> CREATE PROCEDURE update_course_fees(
    ->     IN course_code INT,  -- Input parameter: Course code to update
    ->     IN new_fees FLOAT    -- Input parameter: New fees to set for the course
    -> )
    -> BEGIN
    ->     -- Update the fees for the specified course
    ->     UPDATE COURSE
    ->     SET CRS_FEES = new_fees
    ->     WHERE CRS_CODE = course_code; -- Filter by the provided course code
    -> END$$
Query OK, 0 rows affected (0.025 sec)
```

Call procedure

```
MariaDB [Online_course]> CALL update_course_fees(102, 2000);
Query OK, 1 row affected (0.006 sec)
```

```
MariaDB [Online_course]> select CRS_CODE , CRS_NAME, CRS_FEES from course;
+----------+----------------------------+----------+
| CRS_CODE | CRS_NAME                   | CRS_FEES |
+----------+----------------------------+----------+
|      101 | Calculus I                 |     1500 |
|      102 | Introduction to Programming |     2000 |
|      103 | Physics I                  |     1700 |
|      104 | Macroeconomics             |     1600 |
|      105 | Data Structures            |     2000 |
+----------+----------------------------+----------+
5 rows in set (0.001 sec)
```

Procedure 4: drop student from course;

Drop Student from Course: Removes an enrollment record of a student from a course.

-- Change the delimiter to $$ to allow the creation of the procedure

DELIMITER $$

```
-- Create a stored procedure named `update_course_fees`
-- This procedure updates the fees for a specific course
CREATE PROCEDURE update_course_fees(
    IN course_code INT,  -- Input parameter: Course code to update
    IN new_fees FLOAT    -- Input parameter: New fees to set for the course
)
BEGIN
    -- Update the fees for the specified course
    UPDATE COURSE
    SET CRS_FEES = new_fees
    WHERE CRS_CODE = course_code; -- Filter by the provided course code
END$$

-- Reset the delimiter back to the default semicolon
DELIMITER ;
```

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Change the delimiter to $$ to allow the creation of the procedure
MariaDB [Online_Course]> DELIMITER $$
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Create a stored procedure named `update_course_fees`
MariaDB [Online_Course]> -- This procedure updates the fees for a specific course
MariaDB [Online_Course]> CREATE PROCEDURE update_course_fees(
    ->     IN course_code INT,  -- Input parameter: Course code to update
    ->     IN new_fees FLOAT    -- Input parameter: New fees to set for the course
    -> )
    -> BEGIN
    ->     -- Update the fees for the specified course
    ->     UPDATE COURSE
    ->     SET CRS_FEES = new_fees
    ->     WHERE CRS_CODE = course_code; -- Filter by the provided course code
    -> END$$
ERROR 1304 (42000): PROCEDURE update_course_fees already exists
MariaDB [Online_Course]>
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]>
```

Before drop student

```
MariaDB [Online_course]> select * from enrollment;
+------------+----------+------------+------------------+-------+---------+
| STUDENT_ID | CRS_CODE | ENROL_DATE | COMPLETION_STATE | GRADE | PAYMENT |
+------------+----------+------------+------------------+-------+---------+
|          1 |      101 | 2025-01-10 | In Progress      | NULL  | Paid    |
|          2 |      101 | 2025-01-25 | In Progress      | NULL  | Paid    |
|          2 |      102 | 2025-01-12 | In Progress      | NULL  | Paid    |
|          3 |      102 | 2025-01-25 | Completed        | A     | Paid    |
|          3 |      103 | 2025-01-15 | In Progress      | NULL  | Paid    |
|          4 |      104 | 2025-01-20 | In Progress      | NULL  | Paid    |
|          5 |      105 | 2025-01-25 | In Progress      | NULL  | Paid    |
+------------+----------+------------+------------------+-------+---------+
7 rows in set (0.001 sec)
```

Call procedure

```
MariaDB [Online_course]> CALL drop_student(2, 101);
Query OK, 2 rows affected (0.005 sec)
```

After drop student

```
MariaDB [Online_course]> select * from enrollment;
+------------+----------+------------+------------------+-------+---------+
| STUDENT_ID | CRS_CODE | ENROL_DATE | COMPLETION_STATE | GRADE | PAYMENT |
+------------+----------+------------+------------------+-------+---------+
|          2 |      102 | 2025-01-12 | In Progress      | NULL  | Paid    |
|          3 |      102 | 2025-01-25 | Completed        | A     | Paid    |
|          3 |      103 | 2025-01-15 | In Progress      | NULL  | Paid    |
|          4 |      104 | 2025-01-20 | In Progress      | NULL  | Paid    |
|          5 |      105 | 2025-01-25 | In Progress      | NULL  | Paid    |
+------------+----------+------------+------------------+-------+---------+
5 rows in set (0.001 sec)
```

These procedures simplify common tasks and ensure consistency across the system.

7: TEST:

Results of the implementation can be seen from the run of SQL commands, which are executed in this document, are presented in screenshots and outputs: table creation, insertion of data, trigger activation, function calls, and procedure execution.

Insert the remainder student into enrollment table

```
-- Insert multiple enrollment records into the ENROLLMENT table
INSERT INTO ENROLLMENT (STUDENT_ID, CRS_CODE, ENROL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
VALUES
  -- Enrollment for STUDENT_ID 6 in COURSE 101
  (6, 101, CURDATE(), 'In Progress', NULL, 'Paid'),

  -- Enrollment for STUDENT_ID 7 in COURSE 102
  (7, 102, CURDATE(), 'In Progress', NULL, 'Paid'),

  -- Enrollment for STUDENT_ID 8 in COURSE 103
  (8, 103, CURDATE(), 'In Progress', NULL, 'Paid'),

  -- Enrollment for STUDENT_ID 9 in COURSE 104
  (9, 104, CURDATE(), 'In Progress', NULL, 'Paid'),

  -- Enrollment for STUDENT_ID 10 in COURSE 105
  (10, 105, CURDATE(), 'In Progress', NULL, 'Paid'),

  -- Enrollment for STUDENT_ID 11 in COURSE 101
  (11, 101, CURDATE(), 'In Progress', NULL, 'Paid'),

  -- Enrollment for STUDENT_ID 12 in COURSE 102
  (12, 102, CURDATE(), 'In Progress', NULL, 'Paid'),
```

-- Enrollment for STUDENT_ID 13 in COURSE 103 (Payment status: Unpaid)

(13, 103, CURDATE(), 'In Progress', NULL, 'Unpaid'),

-- Enrollment for STUDENT_ID 14 in COURSE 104 (Payment status: Unpaid)

(14, 104, CURDATE(), 'In Progress', NULL, 'Unpaid'),

-- Enrollment for STUDENT_ID 15 in COURSE 105 (Payment status: Unpaid)

(15, 105, CURDATE(), 'In Progress', NULL, 'Unpaid');

Output

```
MariaDB [Online_Course]> -- Reset the delimiter back to the default semicolon
MariaDB [Online_Course]> DELIMITER ;
MariaDB [Online_Course]> -- Insert multiple enrollment records into the ENROLLMENT table
MariaDB [Online_Course]> INSERT INTO ENROLLMENT (STUDENT_ID, CRS_CODE, ENROL_DATE, COMPLETION_STATE, GRADE, PAYMENT)
    -> VALUES
    ->     -- Enrollment for STUDENT_ID 6 in COURSE 101
    ->     (6, 101, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 7 in COURSE 102
    ->     (7, 102, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 8 in COURSE 103
    ->     (8, 103, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 9 in COURSE 104
    ->     (9, 104, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 10 in COURSE 105
    ->     (10, 105, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 11 in COURSE 101
    ->     (11, 101, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 12 in COURSE 102
    ->     (12, 102, CURDATE(), 'In Progress', NULL, 'Paid'),
    ->
    ->     -- Enrollment for STUDENT_ID 13 in COURSE 103 (Payment status: Unpaid)
    ->     (13, 103, CURDATE(), 'In Progress', NULL, 'Unpaid'),
    ->
    ->     -- Enrollment for STUDENT_ID 14 in COURSE 104 (Payment status: Unpaid)
    ->     (14, 104, CURDATE(), 'In Progress', NULL, 'Unpaid'),
    ->
    ->     -- Enrollment for STUDENT_ID 15 in COURSE 105 (Payment status: Unpaid)
    ->     (15, 105, CURDATE(), 'In Progress', NULL, 'Unpaid');
Query OK, 10 rows affected (0.033 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

<u>Select</u>

```
MariaDB [Online_course]> SELECT S.STUDENT_ID, S.STUDENT_FNAME, S.STUDENT_LNAME, E.CRS_CODE, E.PAYMENT
    -> FROM STUDENT S
    -> JOIN ENROLLMENT E ON S.STUDENT_ID = E.STUDENT_ID
    -> WHERE E.PAYMENT IS NULL OR E.PAYMENT != 'Paid';
+------------+---------------+---------------+----------+---------+
| STUDENT_ID | STUDENT_FNAME | STUDENT_LNAME | CRS_CODE | PAYMENT |
+------------+---------------+---------------+----------+---------+
|         13 | Noura         | Mutairi       |      103 | Unpaid  |
|         14 | Salem         | Qahtani       |      104 | Unpaid  |
|         15 | Jamila        | Anzi          |      105 | Unpaid  |
+------------+---------------+---------------+----------+---------+
3 rows in set (0.001 sec)
```

```
MariaDB [Online_course]> SELECT STUDENT_GENDER, COUNT(*) AS Total
    -> FROM STUDENT
    -> GROUP BY STUDENT_GENDER;
+----------------+-------+
| STUDENT_GENDER | Total |
+----------------+-------+
| F              |     7 |
| M              |     8 |
+----------------+-------+
2 rows in set (0.001 sec)
```

```
MariaDB [Online_course]> SELECT CRS_CODE, COUNT(STUDENT_ID) AS Total_Students
    -> FROM ENROLLMENT
    -> GROUP BY CRS_CODE
    -> HAVING COUNT(STUDENT_ID) > 2;
+----------+----------------+
| CRS_CODE | Total_Students |
+----------+----------------+
|      101 |              3 |
|      102 |              4 |
|      103 |              3 |
|      104 |              3 |
|      105 |              3 |
+----------+----------------+
5 rows in set (0.001 sec)
```

```
MariaDB [online_course]> SELECT * FROM ENROLLMENT WHERE CRS_CODE = 101;
+------------+----------+------------+------------------+-------+---------+
| STUDENT_ID | CRS_CODE | ENROL_DATE | COMPLETION_STATE | GRADE | PAYMENT |
+------------+----------+------------+------------------+-------+---------+
|          1 |      101 | 2025-01-10 | In Progress      | NULL  | Paid    |
|          6 |      101 | 2025-01-26 | In Progress      | NULL  | Paid    |
|         11 |      101 | 2025-01-26 | In Progress      | NULL  | Paid    |
+------------+----------+------------+------------------+-------+---------+
3 rows in set (0.000 sec)
```

```
MariaDB [online_course]> SELECT * FROM ASSIGNMENT WHERE CRS_CODE = 102;
+----------+-------------+-------------------------+----------------+---------+
| ASSIGN_ID | ASSIGN_TITLE | ASSIGN_DESCRIPTION       | ASSIGN_DUE_DATE | CRS_CODE |
+----------+-------------+-------------------------+----------------+---------+
|         2 | Assignment 2 | Python Programming Basics | 2025-02-05      |      102 |
+----------+-------------+-------------------------+----------------+---------+
1 row in set (0.001 sec)
```

```
MariaDB [online_course]> SELECT * FROM SUBMISSION WHERE STUDENT_ID = 1;
+------------+---------------+------------+------------+
| STUDENT_ID | ASSIGNMENT_ID | SUBM_DATE  | SUBM_GRADE |
+------------+---------------+------------+------------+
|          1 |             1 | 2025-01-26 | A          |
+------------+---------------+------------+------------+
1 row in set (0.001 sec)
```

DESCRIBE:

```
MariaDB [online_course]> DESCRIBE STUDENT;
+-----------------+--------------+------+-----+---------+-------+
| Field           | Type         | Null | Key | Default | Extra |
+-----------------+--------------+------+-----+---------+-------+
| STUDENT_ID      | int(11)      | NO   | PRI | NULL    |       |
| STUDENT_FNAME   | varchar(50)  | YES  |     | NULL    |       |
| STUDENT_LNAME   | varchar(50)  | YES  |     | NULL    |       |
| STUDENT_EMAIL   | varchar(100) | YES  |     | NULL    |       |
| STUDENT_PHONE   | varchar(15)  | YES  |     | NULL    |       |
| STUDENT_DOB     | date         | YES  |     | NULL    |       |
| STUDENT_GENDER  | char(1)      | YES  |     | NULL    |       |
| STUDENT_ADDRESS | varchar(255) | YES  |     | NULL    |       |
+-----------------+--------------+------+-----+---------+-------+
8 rows in set (0.005 sec)
```

```
MariaDB [online_course]> DESCRIBE INSTRUCTOR;
+-----------------+--------------+------+-----+---------+-------+
| Field           | Type         | Null | Key | Default | Extra |
+-----------------+--------------+------+-----+---------+-------+
| INSTR_ID        | int(11)      | NO   | PRI | NULL    |       |
| INSTR_NAME      | varchar(100) | YES  |     | NULL    |       |
| INSTR_EMAIL     | varchar(100) | YES  |     | NULL    |       |
| INSTR_PHONE     | varchar(15)  | YES  |     | NULL    |       |
| EXPERTISE_LEVEL | varchar(50)  | YES  |     | NULL    |       |
+-----------------+--------------+------+-----+---------+-------+
5 rows in set (0.003 sec)
```

```
MariaDB [online_course]> DESCRIBE COURSE;
+-----------------+--------------+------+-----+---------+-------+
| Field           | Type         | Null | Key | Default | Extra |
+-----------------+--------------+------+-----+---------+-------+
| CRS_CODE        | int(11)      | NO   | PRI | NULL    |       |
| CRS_NAME        | varchar(100) | YES  |     | NULL    |       |
| CRS_DESCRIPTION | varchar(255) | YES  |     | NULL    |       |
| CRS_TIME        | varchar(50)  | YES  |     | NULL    |       |
| CRS_START_DATE  | date         | YES  |     | NULL    |       |
| CRS_END_DATE    | date         | YES  |     | NULL    |       |
| LECTURE_LINK    | varchar(255) | YES  |     | NULL    |       |
| CRS_FEES        | float        | YES  |     | NULL    |       |
| INSTR_ID        | int(11)      | YES  | MUL | NULL    |       |
+-----------------+--------------+------+-----+---------+-------+
9 rows in set (0.003 sec)
```

```
MariaDB [online_course]> DESCRIBE ENROLLMENT;
+------------------+-------------+------+-----+---------+-------+
| Field            | Type        | Null | Key | Default | Extra |
+------------------+-------------+------+-----+---------+-------+
| STUDENT_ID       | int(11)     | NO   | PRI | NULL    |       |
| CRS_CODE         | int(11)     | NO   | PRI | NULL    |       |
| ENROL_DATE       | date        | YES  |     | NULL    |       |
| COMPLETION_STATE | varchar(50) | YES  |     | NULL    |       |
| GRADE            | varchar(2)  | YES  |     | NULL    |       |
| PAYMENT          | varchar(50) | YES  |     | NULL    |       |
+------------------+-------------+------+-----+---------+-------+
6 rows in set (0.003 sec)
```

```
MariaDB [online_course]> DESCRIBE ASSIGNMENT;
+--------------------+--------------+------+-----+---------+-------+
| Field              | Type         | Null | Key | Default | Extra |
+--------------------+--------------+------+-----+---------+-------+
| ASSIGN_ID          | int(11)      | NO   | PRI | NULL    |       |
| ASSIGN_TITLE       | varchar(100) | YES  |     | NULL    |       |
| ASSIGN_DESCRIPTION | varchar(255) | YES  |     | NULL    |       |
| ASSIGN_DUE_DATE    | date         | YES  |     | NULL    |       |
| CRS_CODE           | int(11)      | YES  | MUL | NULL    |       |
+--------------------+--------------+------+-----+---------+-------+
5 rows in set (0.002 sec)
```

```
MariaDB [online_course]> DESCRIBE SUBMISSION;
+---------------+------------+------+-----+---------+-------+
| Field         | Type       | Null | Key | Default | Extra |
+---------------+------------+------+-----+---------+-------+
| STUDENT_ID    | int(11)    | NO   | PRI | NULL    |       |
| ASSIGNMENT_ID | int(11)    | NO   | PRI | NULL    |       |
| SUBM_DATE     | date       | YES  |     | NULL    |       |
| SUBM_GRADE    | varchar(2) | YES  |     | NULL    |       |
+---------------+------------+------+-----+---------+-------+
4 rows in set (0.003 sec)
```

```
MariaDB [online_course]> DESCRIBE LEARNING_MATERIAL;
+-------------------+--------------+------+-----+---------+-------+
| Field             | Type         | Null | Key | Default | Extra |
+-------------------+--------------+------+-----+---------+-------+
| MATER_ID          | int(11)      | NO   | PRI | NULL    |       |
| MATER_TITLE       | varchar(100) | YES  |     | NULL    |       |
| MATER_TYPE        | varchar(50)  | YES  |     | NULL    |       |
| MATER_UPLOAD_DATE | date         | YES  |     | NULL    |       |
| CRS_CODE          | int(11)      | YES  | MUL | NULL    |       |
| ACCESS_PERMISSION | varchar(50)  | YES  |     | NULL    |       |
+-------------------+--------------+------+-----+---------+-------+
6 rows in set (0.004 sec)
```

Joins

```
MariaDB [online_course]> SELECT
    ->     S.STUDENT_ID,
    ->     S.STUDENT_FNAME,
    ->     S.STUDENT_LNAME,
    ->     E.CRS_CODE,
    ->     E.ENROL_DATE
    -> FROM
    ->     STUDENT S
    -> INNER JOIN
    ->     ENROLLMENT E ON S.STUDENT_ID = E.STUDENT_ID;
+------------+---------------+---------------+----------+------------+
| STUDENT_ID | STUDENT_FNAME | STUDENT_LNAME | CRS_CODE | ENROL_DATE |
+------------+---------------+---------------+----------+------------+
|          1 | Mohammed      | Ali           |      101 | 2025-01-10 |
|          2 | Ahmed         | Saleh         |      102 | 2025-01-12 |
|          3 | Fatima        | Sharif        |      103 | 2025-01-15 |
|          4 | Khalid        | Saeed         |      104 | 2025-01-20 |
|          5 | Noor          | Hashimi       |      105 | 2025-01-25 |
|          6 | Abdullah      | Ghamdi        |      101 | 2025-01-26 |
|          7 | Hind          | Otaibi        |      102 | 2025-01-26 |
|          8 | Badr          | Dosari        |      103 | 2025-01-26 |
|          9 | Layla         | Shahrani      |      104 | 2025-01-26 |
|         10 | Saud          | Zaid          |      105 | 2025-01-26 |
|         11 | Mona          | Khalid        |      101 | 2025-01-26 |
|         12 | Abdulrahman   | Tamimi        |      102 | 2025-01-26 |
|         13 | Noura         | Mutairi       |      103 | 2025-01-26 |
|         14 | Salem         | Qahtani       |      104 | 2025-01-26 |
|         15 | Jamila        | Anzi          |      105 | 2025-01-26 |
+------------+---------------+---------------+----------+------------+
15 rows in set (0.002 sec)
```

```
MariaDB [online_course]> SELECT
    ->     C.CRS_CODE,
    ->     C.CRS_NAME,
    ->     I.INSTR_NAME,
    ->     I.EXPERTISE_LEVEL
    -> FROM
    ->     COURSE C
    -> INNER JOIN
    ->     INSTRUCTOR I ON C.INSTR_ID = I.INSTR_ID;
+----------+-----------------------------+---------------------+----------------------+
| CRS_CODE | CRS_NAME                    | INSTR_NAME          | EXPERTISE_LEVEL      |
+----------+-----------------------------+---------------------+----------------------+
|      101 | Calculus I                  | Dr. Hassan Al-Sayed | Advanced Mathematics |
|      102 | Introduction to Programming | Prof. Layla Mahmoud | Computer Science     |
|      103 | Physics I                   | Dr. Faisal Al-Harbi | Physics              |
|      104 | Macroeconomics              | Dr. Mona Al-Qassem  | Economics            |
|      105 | Data Structures             | Prof. Layla Mahmoud | Computer Science     |
+----------+-----------------------------+---------------------+----------------------+
5 rows in set (0.003 sec)
```

```
MariaDB [online_course]> SELECT
    ->     S.STUDENT_ID,
    ->     S.STUDENT_FNAME,
    ->     S.STUDENT_LNAME,
    ->     E.CRS_CODE
    -> FROM
    ->     STUDENT S
    -> LEFT JOIN
    ->     ENROLLMENT E ON S.STUDENT_ID = E.STUDENT_ID;
+------------+---------------+---------------+----------+
| STUDENT_ID | STUDENT_FNAME | STUDENT_LNAME | CRS_CODE |
+------------+---------------+---------------+----------+
|          1 | Mohammed      | Ali           |      101 |
|          2 | Ahmed         | Saleh         |      102 |
|          3 | Fatima        | Sharif        |      103 |
|          4 | Khalid        | Saeed         |      104 |
|          5 | Noor          | Hashimi       |      105 |
|          6 | Abdullah      | Ghamdi        |      101 |
|          7 | Hind          | Otaibi        |      102 |
|          8 | Badr          | Dosari        |      103 |
|          9 | Layla         | Shahrani      |      104 |
|         10 | Saud          | Zaid          |      105 |
|         11 | Mona          | Khalid        |      101 |
|         12 | Abdulrahman   | Tamimi        |      102 |
|         13 | Noura         | Mutairi       |      103 |
|         14 | Salem         | Qahtani       |      104 |
|         15 | Jamila        | Anzi          |      105 |
+------------+---------------+---------------+----------+
15 rows in set (0.001 sec)
```

```
MariaDB [online_course]> SELECT
    ->     S.STUDENT_ID,
    ->     A.ASSIGN_TITLE,
    ->     C.CRS_NAME,
    ->     S.SUBM_DATE,
    ->     S.SUBM_GRADE
    -> FROM
    ->     SUBMISSION S
    -> INNER JOIN
    ->     ASSIGNMENT A ON S.ASSIGNMENT_ID = A.ASSIGN_ID
    -> INNER JOIN
    ->     COURSE C ON A.CRS_CODE = C.CRS_CODE;
+------------+--------------+------------+------------+------------+
| STUDENT_ID | ASSIGN_TITLE | CRS_NAME   | SUBM_DATE  | SUBM_GRADE |
+------------+--------------+------------+------------+------------+
|          1 | Assignment 1 | Calculus I | 2025-01-26 | A          |
|          2 | Assignment 1 | Calculus I | 2025-01-26 | B          |
|          3 | Assignment 1 | Calculus I | 2025-01-26 | C          |
+------------+--------------+------------+------------+------------+
3 rows in set (0.000 sec)
```

```
MariaDB [online_course]> SELECT
    ->     E.STUDENT_ID,
    ->     S.STUDENT_FNAME,
    ->     S.STUDENT_LNAME,
    ->     C.CRS_NAME,
    ->     E.ENROL_DATE
    -> FROM
    ->     ENROLLMENT E
    -> INNER JOIN
    ->     STUDENT S ON E.STUDENT_ID = S.STUDENT_ID
    -> INNER JOIN
    ->     COURSE C ON E.CRS_CODE = C.CRS_CODE
    -> WHERE
    ->     C.CRS_CODE = 101;  -- Filter by course code
+------------+---------------+---------------+------------+------------+
| STUDENT_ID | STUDENT_FNAME | STUDENT_LNAME | CRS_NAME   | ENROL_DATE |
+------------+---------------+---------------+------------+------------+
|          1 | Mohammed      | Ali           | Calculus I | 2025-01-10 |
|          6 | Abdullah      | Ghamdi        | Calculus I | 2025-01-26 |
|         11 | Mona          | Khalid        | Calculus I | 2025-01-26 |
+------------+---------------+---------------+------------+------------+
3 rows in set (0.001 sec)
```

This will help in assuring that the database is working as expected and all the rules and constraints.

8 Business Ideas

Revolutionizing Education Through Digital Integration

Bin Thabit Hussein

1. **Problem Identification and Solution Alignment**

   Traditional course management systems are riddled with inefficiencies like manual processing, poor monitoring of student progress, and disjointed academic processes. ECMS eliminates all these problems with an integrated database-driven solution providing automatic functionality in critical tasks like course enrollment, assignment tracking, grading, and resource management. This makes education more efficient and accessible for both the instructors and the students.

2. **Strategic Planning**

   Potential Users and Needs

- Students: Require a system to register for courses, track progress, see assignments, and access learning materials. The system provides immediate feedback on performance and deadlines.

- Instructors: Need tools to plan courses, track student progress, create assignments, and provide feedback.

- Administrators: Need a centralized system to manage operations, manage data, and provide consistency.

Promotion and Outreach Promotion will include:

- **Social Media:** Educational professional-targeted campaigns.

- **Partnerships:** Collaboration with institutions to offer free trials.

- **Webinars:** Demos to illustrate the system's advantages.

3. **Implementation Challenges**

SWOT Analysis

- **Strengths:** Centralized management, increased efficiency, scalability, real-time reporting.
- **Weaknesses:** Initial cost is high, users might resist, real-time collaboration has limitations.
- **Opportunities:** Demand for online learning increasing, entering corporate training.
- **Threats:** Competition, fast-paced technological changes, data security.

Potential Challenges

- **Cost:** High initial cost might be prohibitive to small institutions.
  Technical Issues: Compatibility with current systems and scalability issues.
- **User Adoption:** Resistance to change from established ways.
- **Data Security:** Sensitive data must be protected properly.

Through these breakthroughs, the ECMS can transform course management and respond to the evolving needs of educational institutions.

AI-driven Personalized Learning Platform

Name: Jaeran, Osamah Nabil Saleh Ali

1. Problem Identification and Solution Alignment

Problem: Most learners face challenges when trying to decide on courses that best match their skill levels, learning pace, and career goals. Traditional online learning platforms are extremely inclined towards one-size-fits-all approaches, which may cause disengagement and inefficiency regarding learning outcomes.

Solution: The proposed intelligent personalized learning platform shall utilize machine learning algorithms to analyze user behavior, preference, and performance for recommending appropriate courses, learning paths, and resources for each user. In this way, engagement and effective learning shall be increased.

2. Strategic Planning

Potential Users:
- Students: For personalized study plans with better academic performance in view.
- Professional: To advance in careers by adding skills.
- Lifelong Learners: To learn about new topics at their own pace.

Specific Needs:
- Students need structured learning paths to achieve academic goals.
- Professionals require industry-relevant skills and certifications.
- Lifelong learners want flexibility and personalized recommendations.

Promotion Strategy:
- Social Media Marketing: Utilize LinkedIn, Instagram, and YouTube to showcase the AI-driven features of the platform and success stories.
- Partnerships: Partner with universities, corporations, and influencers in the education space to promote the platform.
- Free Trials: Direct free trials that will show them how the personalized learning actually works.

3. Implementation Challenges

Strengths:
Personalization:
- AI-powered recommendations for lessons, activities, or resources for improving user learning outcomes and experience.

Scability:
- Platform designed to host thousands of users at relatively low incremental costs.

Competitive Advantage:
- Distinction of AI functionality in the online learning environment.

Weaknesses:
High Development Costs:
- Very expensive to both develop and maintain AI algorithms.

Data Privacy:
- Users' unwillingness to share personal data for AI analysis.

Possible Challenges:
Technical Pre-requisites:

- For crafting proper and result-oriented AI algorithms, machine learning and data science knowledge is needed.

User Adoption:

- It may take some time to have users up and running, believing an AI-driven system.

Cost:

- Initial Investment in AI Development and Infrastructure: The cost can be prohibitively high.

Blockchain-Based Digital Identity Verification Platform

YOUSEF.RIAN ALI

1. Problem Identification and Solution Alignme

Identity theft and data breaches are a widespread issue with financial loss and loss of trust in virtual environments

Solution:

A Blockchain-Based Digital Identification Platform that

1 Uses blockchain technology for secure, decentralized creation of digital identity.

2. Gives users control over and management of their information and access permission when desired.

3. Includes multi-factor authentication for increased security.

4. Includes real-time, tamper-evidence verification for companies, governments, and citizens.

2. Strategic Planning

Target Users:

Governments: To issue secure digital IDs and passports

Financial Institutions: To comply with KYC Know Your Customer and AML Anti-Money Laundering requirements

E-commerce platforms: To verify user identity and prevent fraud

Individual: To secure online identity and private information

User Needs:

Government: Stop identity fraud and make ID issuance easier.

Business: Optimize onboarding and verification processes and preserve security.

Individual: Own information and prevent misuse of private information.

Promotion Strategy:

Partner with governments and large companies for pilot programs.

Emphasize security, privacy, and cost-effectiveness of platform in marketing.

Leverage industry events and conventions to expose technology capabilities.

Implementation Challenges

3. Implementation Challenges

Strengths:

Secure, tamper-evidence, and decentralized system for safeguarding.

- User-managed information for increased confidence and compliance with data protection legislation.

- Scalable in industries and use cases

Weaknesses:

High development cost for integration with blockchain.
- Requires training for effective use of technology.

SWOT Analysis:

Strengths: Secure tamper-evidence and usermanaged identity.
Weaknesses: High initial cost and complexity
Opportunities: Broadened demand for secure digital identity and increased regulatory pressure for increased security for information
Threats: competition in concentrated identity management solutions and evolving regulation of blockchain.

Potential Challenges:

Cost: infrastructure development and maintenance via blockchain.

Regulation: compliance with international privacy and security laws.

Adoption: convincing users and institutions to switch to a blockchain-based system.

Online Learning Platform for Small and Medium-Sized Educational Institutions

Sharafuddin Mohammed Abdulsalam Ahmed

Problem Identification and Solution Alignment

How does your system solve a real-world problem?

Small and medium-sized educational organizations cannot afford to invest in a powerful online learning administration system. They are affected by a number of problems like:

- Insufficient budgets for costly e-learning applications.
- Difficulty in student data, course material management, and tracking progress.
- Lack of personalized learning experience for students.

Solution:

A moderately priced, friendly-to-use, scalable eLearning Web portal that befits the needs and requirements of a small and medium establishment. Services proposed include:

- ***Central Database:*** Store all student data along with courseware and progress tracking in one database.
- ***Personalised learning paths:*** Customized learning recommended automatically on the basis of individual performance enabled by AI learning.
- ***Interactive Element:*** Live streaming of classes along with quizzes and discussion forums facilitate heightened interest from students.
- ***Analytics Dashboard:*** Reporting students' performance and course effectiveness will be reported to teachers and administrators.
- ***Alignment:*** Solution directly solves the problem by means of an affordable, scalable, and feature-rich solution to improve the access, engagement, and management of data in the institutions with a relatively small scale.

Strategic Planning:

Who is your target audience in terms of possible users, and what are their very specific needs?

- ***Primary:*** Small and medium-sized schools, colleges, and training centers.

  ***Needs*:** Affordable, easy-to-use, and customizable web-based learning tools.
- ***Secondary Users:*** Teachers and administrators.

  ***Needs*:** Course development, student monitoring, and performance analysis.
- ***Consumers*:** The students.

  ***Needs*:** Interactive and personalized learning.

How would you market your system and reach the potential users?

- **Partnerships:** Partner with educational associations and government programs targeting small institutions.
- **Freemium Model:** Provide a free basic version, and then charge money for premium features, depending on the requirements of advanced needs.
- **Digital Marketing:** Run targeted ads on LinkedIn, Facebook, and Google targeting educators and administrators.
- **Webinars/Demos:** Provide webinars free of cost to showcase the features of the platform.
- **Referral Program:** Encourage referrals from existing users.

Implementation Challenges

What are your system's strengths, and what are the weaknesses?

Strengths:

- Affordability and scalability for small institutions.
- A user-friendly interface for nontechnical users.

- AI-driven personalization is a plus for learner success.
- A unified-back-end database simplifies data management.

Counterparts:

- Low brand awareness as compared to recognized leaders in the industry.
- The need for the internet, which can be very challenging in rural areas
- Must comply with new trends in education and tech.

What are the rollout challenges that may impact a successful implementation?

- ***Cost*:** Platform development and upkeep are at a large upfront cost.
- ***Technical Environment*:** Interoperability with other devices and operating systems.
- ***Acceptance*:** Moving institutions from traditional, pen-and-paper teaching to an online environment.
- ***Data Security***: Safeguarding student and institution data privacy against any form of threat.

Thwart Strategies:

- Private treaties that match customers' pockets.
- More rigorous testing has to be in place for interoperability and usability.
- User training and support must be available to ease the transition.
- **Data Security:** Include encryption and implement several security protocols.

Corporate Training and Development Platform

Sharafuddin Mohammed Abdulsalam Ahmed

Problem Identification and Solution Alignment

How do you solve a practical problem with your system?

This has been a challenge to many companies when it comes to employee training and development because of:

- Expensive training programs that employ the traditional ways of training.
- The inability to personalize learning paths for employees.
- Tracking the progress as well as the skill gaps of the employees.
- Rigid training schedules that waste time.

Solution:

We will provide solutions to these corporate trainings that are flexible, affordable and truly data driven. The main features are as follows:

- ***Tools for Skill Assessment*:** Gaps in skills identification, and training programs tailored to the needs recommended
- ***Microlearning Modules:*** Short lessons focused on a specific topic which employees can take when it is convenient to them.
- ***Tracking Progress:*** Managers are able to monitor an employee's work performance and development with sophisticated technology.
- ***Gamification:*** Earning badges, being on a leader board, and other rewards-slickened disengagement.
- ***Integration with HR Systems:*** Data management with other HR systems will be easier because of the platform's seamless integration.
- ***Alignment:*** The key components of the system speak to the problem because they increase the level of employee skills and performance while scaling the

organization in a way that is efficient, engaging, and tailored to the employees' training needs.

Strategic Planning

Who is your system for, and what are their particular needs?

- ***Primary Users:*** HR departments and training managers in corporations.

  Needs: Affordable, scalable, and measurable training solutions.
- ***Secondary Users***: Employees.

  ***Needs*:** Training content that is flexible, engaging, and relevant.
- ***End Users***: Executives and decision-makers.

  ***Needs*:** Data-driven insights into the ROI of training programs.

How would you market your system and get the word out to potential users?

- ***B2B Marketing*:** Targeting HR professionals with LinkedIn ads, industry conferences, and trade publications.
- ***Free Trials*:** Conceive a 30-day free trial with the intention of showcasing value to prospects.
- ***Case Studies and Testimonials***: Publish success stories of early adopters to build credibility within the market.
- ***Partnerships:*** Collaboration with HR software providers and industry associations
- ***Content Marketing:*** Create and publish blogs, whitepapers, and webinars on corporate training trends.

Implementation Challenges

What are the scenarios for the system, s and Was?

Pros:

- An affordable price when compared to the conventional methodology of staff training.

- The personal learning expedition actually makes the staff zealous in their classes as well as does well in performance.
- The entire implementation could run effortlessly well under real-time monitoring.
- Game mechanics play a vital role in enhancing motivation, completion rates, and additional crucial performance indicators.

Cons:

- Rising too many voices namely HR, managers, employees.
- The system finds itself in a stand-off against established E-learning platforms, as set forth by the critics.
- An over-customization effect may make the system extremely sophisticated and hence probably a point too far for real-world enterprise implementation.

What are the boundaries? which might hinder an gaining establishing opportunity for probation?

- ***Cost:*** High development and customization costs are involved in introducing the system.
- ***Tech issues:*** Users may face issues such as poor software or hardware, leading audiences towards rewards issues.
- ***User Resistance:*** One primary barrier to adopting something new is likely to be the inability of most litter bearers of the old, multiple-system resistors to adopt new technology.
- ***Content quality:*** Provision of quality information for training materials that will be on-time upkeep.

Some options to block impeding confidence in future current solution postulate:

- Setup a two-level pricing plan that caters to both small and burgeoning enterprises and industry mammoths.
- Put in a robust structure and test the reliability of security.
- Make sure new customers are comfortable and train them well.

9 Conclusion

In the creation and development of E-Learning Course Management System (ECMS), the focusing aspects concerning the design of ECMS end in achieving some important objectives. For instance, we were able to implement a data-driven solution for management of core entities-Courses, Students, Instructors, Assignments, and Material-for ensuring organized data, its accessibility, and validity. The ECMS systems optimize on the issue of improving administrative efficiency since major processes are automated including providing a centralized point for course registration, assignment submission, grading, and materials management. It also enables students to easily check their results and teachers to create course content and evaluate student performance accordingly. The database design incorporates bidirectional linking, which guarantees the integrity of data to minimize redundancy and facilitate smooth academic operations.

The ECMS has major strengths in centralizing management of academic data, automating activities tied to allocating assignments, and offering seamless engagement with the system for both students and instructors. The features deliver in real-time, such as analysis of performance and course management, make it incredibly responsive and useful. Moreover, extensibility and best practices in the industry would assure catering to the expanding requirements of the education industry. Nonetheless, certain restrictions apply to the kind of application this system can become subjected to. It may deteriorate significantly when it experiences high data rates from the volume of users or courses especially without extra optimizations. Also, a small trade-off of using the old relational database model is constrained for media handling, mostly unstructured data. Furthermore, even now, with just backing the basic support of administrative procedures, the system fails to furnish more advanced services like real-time collaboration or interactive learning tools.

In the future, we will be optimizing the system in several ways. First on the list will be performance optimization, where we will be implementing techniques like indexing and database sharding to enable larger datasets to be processed more efficiently. Real-time functionality like notifications, live chats, or collaborative apps can be added to enhance the learning experience further. Moreover, the integration of NoSQL databases will resolve multimedia content concerns by allowingfor the more efficient management of unstructured data. Security will be augmented through encryption and multi-factor authentication to ensure sensitive data protection. Lastly, advanced analytics and reporting features will be integrated, offering better insights into student performance and course performance. These improvements will make the ECMS a more comprehensive, interactive, and secure system for both instructors and learners.

10 references

1. Allen, I. E., & Seaman, J. (2017). *Digital learning compass: Distance education enrollment report 2017*. Babson Survey Research Group. https://onlinelearningsurvey.com/reports/digtiallearningcompassenrollment2017.pdf

2. Means, B., Toyama, Y., Murphy, R., Bakia, M., & Jones, K. (2010). *Evaluation of evidence-based practices in online learning: A meta-analysis and review of online learning studies*. U.S. Department of Education. https://www2.ed.gov/rschstat/eval/tech/evidence-based-practices/finalreport.pdf

3. Siemens, G., & Long, P. (2011). *Penetrating the fog: Analytics in learning and education*. EDUCAUSE Review, 46(5), 30-40. https://er.educause.edu/articles/2011/9/penetrating-the-fog-analytics-in-learning-and-education

4. Picciano, A. G. (2017). *Theories and frameworks for online education: Seeking an integrated model*. Online Learning, 21(3), 166-190. https://olj.onlinelearningconsortium.org/index.php/olj/article/view/1225

5. Noe, R. A., Clarke, A. D. M., & Klein, H. J. (2014). *Learning in the twenty-first-century workplace*. Annual Review of Organizational Psychology and Organizational Behavior, 1, 245-275. https://www.annualreviews.org/doi/10.1146/annurev-orgpsych-031413-091321

6. Bersin, J. (2017). *The disruption of digital learning: Ten things we have learned*. Josh Bersin. https://joshbersin.com/2017/03/the-disruption-of-digital-learning-ten-things-we-have-learned/

7. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson. https://www.pearson.com/store/p/fundamentals-of-database-systems/P100001672473/9780133970777

8. Date, C. J. (2003). *An introduction to database systems* (8th ed.). Addison-Wesley. https://www.pearson.com/store/p/an-introduction-to-database-systems/P100000354637/9780321197849

9. SQL Tutorial - W3Schools. (n.d.). *SQL tutorial*. W3Schools. https://www.w3schools.com/sql/

10. Picciano, A. G. (2019). *Theories and frameworks for online education: Seeking an integrated model*. Educational Technology Research and Development, 67(6), 1689–1703.https://files.eric.ed.gov/fulltext/EJ1154117.pdf

11. Özsu, M. T., & Valduriez, P. (2020). *Principles of distributed database systems* (4th ed.). Springer. https://link.springer.com/book/10.1007/978-3-030-26253-2

12. Olanrewaju, O. O., Oluwatope, A. A., Olaniyi, O. O., & Olayemi, O. O. (n.d.). Design and implementation of an online course management system. Scientific Research Publishing. Retrieved February 27, 2019, from https://www.scirp.org/journal/paperinformation?paperid=91173